

Publicly Verifiable Generalized Secret Sharing Schemes and Their Applications

Liang Zhang, Dongliang Cai, Tao Liu, Xingyu Wu, Haibin Kan*, *Member, IEEE*, and Jiheng Zhang*

Abstract

Generalized secret sharing (GSS) enables flexible access control in distributed systems by allowing secrets to be shared across arbitrary monotone access structures. However, its adoption in transparent and trustless environments is hindered due to the reliance on trusted participants and secure communication channels. This reliance restricts GSS's ability to provide flexible control in the presence of adversaries. In this paper, we propose publicly verifiable generalized secret sharing (PVGSS), a scheme that allows to distribute a secret using generalized access structures while enabling non-interactive public verifiability of participant honesty. PVGSS scheme offers significant potential to advance the fields of blockchain and applied cryptography, such as attribute-based cryptosystem, fine-grained MPC and on-chain secret escrow. Intuitively, we first build two GSS schemes by leveraging recursive Shamir secret sharing and linear secret sharing scheme. Then, we encrypt GSS shares and generate the corresponding non-interactive zero-knowledge proofs. Furthermore, we innovatively apply the GSS reconstruction algorithm to show all encrypted shares bind to the dealer's secret, with only $O(|\mathbb{A}|)$ verification complexity, where $|\mathbb{A}|$ is the number of leaf nodes in the access structure $\mathbb{A}$. To exemplify the practical applicability, we implement a decentralized exchange (DEX) protocol, where fairness and accountable arbitration are considered. Our benchmarks on the BN128 curve demonstrate the computational efficiency of PVGSS schemes, while Ethereum gas cost analysis confirms the viability of the DEX implementation.

Index Terms

Generalized secret sharing, Public verifiability, PVGSS, DEX

I. INTRODUCTION

SECRET sharing (SS) [1] is a foundational primitive in distributed systems, including threshold cryptosystems [4], distributed storage [6], federated learning [5], and secure multi-party computation (MPC) [7]. Over the years, several important properties and generalizations of secret sharing have been developed to address diverse application needs.

One key property is verifiability. Verifiable secret sharing (VSS) [10], [71], [15] allows shareholders to verify the consistency of their shares, while publicly verifiable secret sharing (PVSS) [16], [13], [12] extends this guarantee to any external observer. These mechanisms are crucial in protocols such as distributed key generation [2], [12], randomness beacons [3], [13], and Byzantine fault-tolerant consensus [8].

Another dimension concerns the expressiveness of the access structure. While classic Shamir's SS scheme realizes a simple (t, n) -threshold policy, generalized secret sharing (GSS) [17], [18], [29] supports an arbitrary monotone access structure or a generalized access structure (GAS). GSS represents the general model of secret sharing, where every unauthorized set learns nothing about the secret, and every authorized set can reconstruct it. GAS encompasses and extends beyond (t, n) -threshold schemes, as well as hierarchical [44], [45], weighted [38], [21], and disjunctive/conjunctive structures [37]. Therefore, GSS has diverse applications across various fields, including distributed systems with flexible access control [19], [20]; customizable multi-signature schemes [21]; MPC with complex rules for joint function computation [21]; and authorization management implementing hierarchical access policies [22].

A third important property is linearity. In a linear secret sharing scheme (LSSS) [9], [46], [29], shares are linear combinations of the secret and random field elements. LSSS schemes are equivalent to monotone span programs [9] and are widely used in ciphertext-policy attribute-based encryption (CPABE) [23], [24].

To illustrate their mechanisms: VSS typically uses cryptographic commitments to bind shares to the secret [10], [15], [14], [11]; PVSS combines public-key encryption with non-interactive zero-knowledge (NIZK) proofs for public auditability [16], [13], [12]; and GSS can be constructed by composing multiple threshold schemes [18]. Notably, these properties—GAS, linearity, and verifiability—are not mutually exclusive. For instance, an LSSS can implement a GSS (as long as the access structure is monotone), and VSS protocols can be built on top of LSSS or threshold schemes alike. Thus, modern secret sharing constructions often combine multiple of these features to meet the demands of complex distributed environments.

Liang Zhang and Jiheng Zhang with Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology. (e-mail: scottzhang@ust.hk; jiheng@ust.hk)

Dongliang Cai and Haibin Kan are with School of Computer Science, Fudan University, Shanghai 200433, China and Shanghai Engineering Research Center of Blockchain, Shanghai 200433, China. (e-mail: dlcai22@m.fudan.edu.cn; hbkan@fudan.edu.cn)

Tao Liu and Xingyu Wu are with School of Cyberspace Security (School of Cryptology), Hainan University, Haikou, 570228, China. (e-mail: jib8181632@gmail.com; xingyuwu@hainanu.edu.cn)

*Corresponding author

Despite their strengths, existing approaches have limitations in transparent, trustless settings with versatile access control: (i) VSS and PVSS are typically designed for threshold structures and do not natively support generalized access policies; (ii) VSS, GSS, and LSSS generally assume private channels for share distribution; and (iii) CPABE lacks built-in mechanisms to ensure honest behavior by encryptors or authorities. These gaps highlight the need for a publicly verifiable secret sharing scheme that supports generalized access structures—a primitive essential for next-generation decentralized systems.

TABLE I: Comparison of SS-related cryptographic primitives

Scheme	Verifiable dealer/encryptor	Verifiable shareholders/authorities	Public channel	Public verifiability	GAS-supported
SS	✗	✗	✗	✗	✗
VSS	✓	✓	✗	✗	✗
PVSS	✓	✓	✓	✓	✗
CPABE	✗	✗	✓	✗	✓
LSSS	✗	✗	✗	✗	✓
GSS	✗	✗	✗	✗	✓
PVGSS	✓	✓	✓	✓	✓

In this paper, we propose publicly verifiable generalized secret sharing (PVGSS), which not only supports generalized access structure but also enables public verifiability for both dealers and shareholders. Table I summarizes a comparison of the functional capabilities by different cryptographic primitives. The primary challenge is designing non-interactive zero-knowledge (NIZK) proofs that ensure encrypted shares $\{C_i\}$ bind to the secret s . Firstly, we encrypt each share s_i as $C_i = \text{pk}_i^{s_i}$, where pk_i is the public key of shareholder P_i . Next, we employ the Sigma protocol [42] and the Fiat-Shamir heuristic [43] to generate NIZK proofs for each share.¹ Importantly, we discover that partial data (i.e., $\{\hat{s}_i\}$, or the response values in Sigma protocol) in the proofs can be input parameters of Lagrange interpolations, leveraging the homomorphism of polynomials. Finally, we identify that these Lagrange interpolations actually form the GSSRecon algorithm (Definition 5), which is then utilized to prove the binding of all shares and the secret $S = g^s$. Besides, the proof of binding method also applies to the LSSS-based PVGSS solution. Therefore, we present two implementations of PVGSS schemes in this work. Our solutions emphasizes that GSSRecon cannot be leveraged without properly designed NIZK proofs.

A. Applications

We contend that PVGSS holds significant promise in the field of blockchain and applied cryptography. In this section, we outline potential application scenarios that PVGSS schemes can be utilized. In Section VI, we will dive deep into a decentralized exchange (DEX) protocol applying PVGSS.

Attribute-based cryptosystem. PVGSS can significantly enhance attribute-based cryptosystems [23], [24], [26] by ensuring trust and transparency in key distribution and access control. For example, if PVGSS is leveraged to replace LSSS in CPABE, authorities will be accountable in issuing keys. Additionally, PVGSS allows CPABE encryptors to outsource encryption to untrustworthy servers.

Fine-grained MPC. Participants have equal rights in traditional PVSS based MPC protocols, such as distributed key generation [2] and federated learning [5]. With PVGSS, it will be possible to investigate MPC protocols with fine-grained access control policy, going beyond the capabilities of weighted MPC [21].

Blockchain oracle. Blockchains and smart contracts typically cannot access external information directly, so oracles [61] provide the necessary data feed, such as price feeds, weather data, or other real-world events. PVGSS-based solution can enhance decentralization and public verifiability.

Layer 2. Layer 2 [65] refers to a set of technologies built on top of a blockchain’s base layer to address scalability issues, increase transaction throughput, and reduce costs without compromising the security and decentralization. Essentially, Layer 2 solutions help blockchain networks scale by processing transactions off-chain. PVGSS can be incorporated to establish bridges [62], relay networks [63] or channels [64].

On-chain secret escrow. PVGSS can be effectively used in on-chain secret escrow systems [66], [67], [68] to provide secure and verifiable management of secrets. In such systems, PVGSS allows participants to securely share a secret (e.g., a decryption key) with a designated access structure, ensuring that the secret can only be accessed under specified conditions.

B. Contributions

- We introduce the concept of publicly verifiable generalized secret sharing (PVGSS), which extends the family of secret sharing (SS) schemes. PVGSS is not simply a combination of PVSS instances, making it a non-trivial and challenging problem to design and implement.

¹Similar to the Albatross PVSS scheme [48], we also employ the Sigma protocol and the Fiat-Shamir heuristic to obtain NIZK proofs. However, since PVGSS is not simply a combination of multiple PVSS instances, it diverges significantly from Albatross—as detailed in Section V-D.

- We explicitly demonstrate that LSSS is a specific instance of the GSS scheme. Further, we present two PVGSS constructions by combining NIZK proofs and GSS schemes, which is derived from either recursive Shamir secret sharing or LSSS. We contend that new PVGSS constructions will be of independent interest.
- A unified method—GSSRecon—is carefully integrated into the share verification algorithm. This allows the dealer to prove the correctness of secret distribution under any monotone access structure, showing the simplicity and efficiency of our constructions.
- We formally define the syntax and security properties of PVGSS. The proposed schemes are rigorously analyzed under the Decisional Diffie-Hellman (DDH) assumption, ensuring strong theoretical guarantees against adversarial attacks. We consider an adversary corrupting partial shares from a shareholder in Section V-E, highlighting the fine-grained feature of PVGSS.
- PVGSS has broad applications across various domains. To demonstrate its practical utility, we apply it to construct a decentralized exchange (DEX) protocol, where both fairness in asset trading and accountable arbitration are achieved without relying on a trusted third party.

C. Paper Organization

Section I introduces the motivation, application scenarios, and main contributions of PVGSS. Section II surveys related work, including threshold-based and generalized secret sharing, as well as publicly verifiable systems, positioning PVGSS within the broader cryptographic landscape. Section III provides necessary preliminaries on access structures, Shamir and LSSS-based secret sharing, and Sigma protocols. Section IV formally defines the Generalized Secret Sharing (GSS) scheme and presents two constructions—Shamir-based and LSSS-based—along with their correctness and secrecy properties. Section V introduces the full PVGSS scheme, detailing its syntax, algorithms, and security model. It also formally analyzes the proposed constructions under the DDH assumption. Section VI demonstrates a practical application of PVGSS in building a decentralized exchange (DEX) that ensures fair token swaps and accountable arbitration. Section VII reports experimental results, including monetary cost on Ethereum blockchain. Section VIII concludes the paper and outlines future work. The appendix summarizes the global states using smart contract pseudocode.

II. RELATED WORKS

A. Access Structures in Secret Sharing

As is well known, traditional SS, VSS, and PVSS protocols [1], [10], [16], [13], [12] are designed around the threshold access structure (TAS). Benaloh et al. [18] introduced the concept of generalized secret sharing (GSS) based on recursive threshold secret sharing, where GAS is inherently implied. Beyond schemes based on TAS or GAS, some other protocols have explored specific types of access control mechanisms.

Particularly, Simmons [41] points out that real-world applications require more versatile capabilities for secret sharing. In weighted secret sharing (WSS) [38], [21], each participant is assigned a positive weight. Only if the sum of weights of some participants exceeds the predefined threshold value, can the secret be reconstructed.

In multipartite secret sharing (MSS) [39], [40], an access structure defined over n participants is partitioned into disjoint $m (\ll n)$ compartments, where participants within the same compartment have equal weights. In MSS, each compartment must achieve a specific internal agreement before contributing to the reconstruction of the secret. Tassa et al. [40] point out that any GAS can be regarded as a multipartite access structure with singleton compartments. Hence, reconstructing the secret in MSS scheme depends only on the number of participants from each compartment, rather than their individual identities. In this way, practical applications can focus on m compartments, where TAS is applied, instead of n individuals. However, participants may be designed in different access structures in a single system, resulting in different divisions of disjoint compartments.

Hierarchical secret sharing (HSS) [44], [45], also referred as multilevel secret sharing [41], [44], places participants in disjoint levels according to their importance. HSS can be applied in managing staff in big companies, where employees are assigned to different levels with different privileges. HSS is categorized into disjunctive and conjunctive types [37]. Further, Drăgan et al. [37] introduce the distributive weighted threshold secret sharing, where participants at the same level have the same weight, and the threshold values are 1.

Other related works, such as proactive secret sharing [71] and multiple secret sharing [25], can be regarded as extensions of SS. Moreover, these concepts can also be incorporated into our GSS and PVGSS schemes.

B. Public Verifiability

Public verifiability is a property that allows any third party to verify the correctness of a computation or data without accessing private information. It is crucial in systems like blockchain [28], e-voting [30], fair exchange [53], [54], and secure MPC [31], where transparency and trust are essential. Public verifiability enhances accountability, reduces reliance on trusted parties, and ensures integrity in decentralized environments.

Blockchain's momentum over the past decade stems from its ability to ensure transparency, decentralization and smart contracts. Transactions on the blockchain are considered publicly verifiable, as the operations within these transactions are replicated across distributed nodes. To further safeguard privacy of private keys and sensitive information while executing transactions in a transparent environment, various cryptographic tools are widely adopted.

Merkle tree [32] and accumulators [33] are often employed to prove membership of specific data in a set publicly. Digital signature schemes [34], [52] are leveraged to achieve authentication, i.e., enabling public verifiability of identities. Verifiable random function (VRF) [27] is a cryptographic primitive that generates pseudorandom outputs along with a proof, allowing anyone to verify the correctness of the output without revealing the secret key.

NIZK systems, such as Fiat-Shamir heuristic-based Sigma protocol [42], [43] and zkSNARK [35], are often adopted to prove knowledge of plaintext or correctness of operations in a public manner. For instance, the discrete logarithm equality algorithm, based on Sigma protocol, allows a prover to convince any verifier that two group elements share the same exponentiation. Similarly, zkRollup is a scaling solution for blockchains that leverages zkSNARK to bundle multiple transactions into a single proof [36]. By processing transactions off-chain and submitting only a compact zkSNARK proof to the main chain, zkRollups drastically reduce data load and gas costs while ensuring security and privacy.

C. (Publicly) Verifiable Secret Sharing

Dealer in the SS scheme may distribute invalid shares to deviate from the protocol. To mitigate this problem, VSS [10], [15] ensures that participants can verify the validity of the shares distributed by the dealer, while PVSS [16], [13] extends this by allowing any external observer to verify the integrity of the shares.

The first VSS scheme, proposed by Feldman [10], enhances Shamir secret sharing by adding public commitments to ensure share validity. Shareholders verify their shares against these commitments, ensuring the dealer's honesty without revealing the secret. Recently, polynomial commitments [14], [15], [11] are widely adopted to design new VSS schemes. The secret shares of VSS should not be transferred publicly.

Stadler [16] put forward the first PVSS scheme in 1996. Intuitively, PVSS can be achieved by integrating public key encryption into VSS schemes, allowing secret shares to be delivered and verified on public channels. For example, ElGamal, RSA, and Pailler cryptosystems are utilized in [16], [50], [49], and [51], respectively. Early PVSS schemes require $O(nt)$ verification complexity, where n and t are the amount of shareholders and threshold value, respectively. In 2017, Cascudo et al. [13] introduced SCRAPE PVSS which for the first time achieves $O(n)$ verification complexity. As the core technique, Reed-Solomon code is employed to make all encrypted shares be verified in a linear operation.

Recently, Cascudo et al. successively proposed ALBATROSS [48], HEPVSS/DHPVSS [47], and qCLPVSS [12], which all maintain $O(n)$ verification complexity. ALBATROSS [48] leverages low-degree exponent interpolation to generate NIZK proof, which is essentially acquired from Sigma protocol and Fiat-Shamir heuristic. HEPVSS[47] also uses ElGamal to encrypt the secret share, while DHPVSS[47] leverages Shamir secret sharing on group $\mathbb{G}$ to hide shares. qCLPVSS [12] for the first time enables users to recover the original secret $s \in \mathbb{Z}_q$ by leveraging class group. Recovering the original secret s can make PVSS applicable in more wider applications, such as distributed key generation [2], [12] and MPC [12], [47] protocols.

III. PRELIMINARIES

In this paper, $\mathbb{Z}_q$ denotes a finite field with prime order q , and $\mathbb{G}$ represents a cyclic group of the same order generated by g .

A. Decisional Diffie-Hellman (DDH) Assumption

The Decisional Diffie-Hellman (DDH) [13] assumption states that (g, g^a, g^b, g^{ab}) and (g, g^a, g^b, g^c) are computationally indistinguishable, where $a, b, c \xleftarrow{R} \mathbb{Z}_q$ are chosen uniformly at random. Formally, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ in distinguishing between the two distributions is negligible:

$$|\Pr[\mathcal{A}(g, g^a, g^b, g^{ab}) = 1] - \Pr[\mathcal{A}(g, g^a, g^b, g^c) = 1]| \leq \text{negl}(\lambda),$$

where λ is the security parameter, and $\text{negl}(\lambda)$ denotes a negligible function in λ .

B. Generalized Access Structure (GAS)

Definition 1 (GAS [18], [24]). Let $\mathcal{P} = \{P_1, \dots, P_n\}$ be a set of parties. A collection $\mathbb{A} \in 2^{\mathcal{P}}$ is a generalized access structure (GAS, commonly abbreviated as AS in previous research [18], [24]), if it meets the following two conditions:

- *Non-triviality*: if $B \in \mathbb{A}$, then $|B| \neq 0$.
- *Monotonicity*: if $B \in \mathbb{A}$, $B \subseteq C$, then $C \in \mathbb{A}$.

If $B \in \mathbb{A}$, we say $\mathbb{A}$ is satisfied by B or B is an authorized set; Otherwise, $\mathbb{A}$ is not satisfied by B or B is unauthorized.

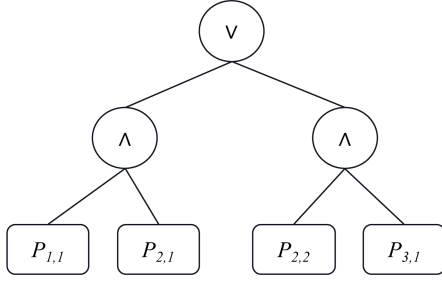


Fig. 1: An access structure using boolean formula, where $\{P_{1,1}, P_{2,1}\}$ and $\{P_{2,2}, P_{3,1}\}$ are authorized sets.

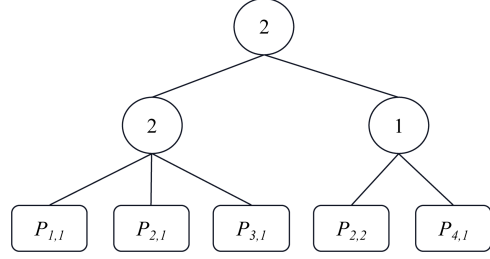


Fig. 2: An access structure using threshold gate, where $\{P_{3,1}, P_{2,2}\}$ and $\{P_{1,1}, P_{3,1}, P_{4,1}\}$ are authorized sets.

For example, $(P_{1,1} \wedge P_{2,1}) \vee (P_{2,2} \wedge P_{3,1})$ and 2 of $(2 \text{ of } (P_{1,1}, P_{2,1}, P_{3,1}), (1 \text{ of } (P_{1,2}, P_{4,1})))$ are access structures, as shown by Figure 1 and Figure 2, where $P_{i,j}$ represents the j th authorization granted to party P_i and $*$ is a wildcard. The disjunction ($\vee$) and conjunction ($\wedge$) in a boolean formula based access structure (cf. Figure 1) is equal to the gate value of 1 and 2 in a threshold-gate based access structure (cf. Figure 2), respectively. Hence, we only consider threshold-gate based access structure in this paper.

Below, we present several of our observations and summarize them as corollaries. These corollaries can be derived by following the definition of GAS.

Corollary 1. *The boolean or threshold-gates based access structure can be regarded as a proposition, which outputs **true** given an authorized set and outputs **false** otherwise.* $\square$

Corollary 2. *An access structure $\mathbb{A}$ can be regarded as a recursive data structure, meaning that any node x in $\mathbb{A}$ with its children nodes is also an access structure, denoted by $\mathbb{A}_x$. For example, there are 8 sub access structures in $\mathbb{A}$ of Figure 2.* $\square$

Corollary 3. *If $B \in \mathbb{A}$, there exists a path (i.e., a leaf node to the root node) in which each sub access structure is satisfied by B . We call it a satisfied path.* $\square$

C. Shamir Secret Sharing (SS)

Definition 2 (Shamir SS). *Shamir secret sharing (SS) scheme allows a dealer to split a secret into n shares for n shareholders. With only a threshold t ($\leq n$) shares, the secret can be successfully recovered, while fewer than t shares reveal nothing about the secret. Shamir SS is defined with following two algorithms.*

- $\{s_1, s_2, \dots, s_n\} \leftarrow \text{Share}(s, n, t)$: Inputs a secret $s \in \mathbb{Z}_q$, n and t , chooses a polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i$, where $a_i \xleftarrow{R} \mathbb{Z}_q$. Finally, the algorithm outputs n shares $f(i)$, $i \in [1, n]$.
- $s \leftarrow \text{Recon}(Q)$: Inputs any t shares $Q \subseteq \{s_1, s_2, \dots, s_n\}$, reconstructs the secret s via the Lagrange interpolation.

Definition 3 (Shamir SS on Group $\mathbb{G}$). *Definition 2 has introduced the original Shamir secret sharing scheme, where a secret $s \in \mathbb{Z}_q$ is shared and recovered. Here we consider how to share secret $S \in \mathbb{G}$ and the dealer does not know the value s such that $S = g^s$. The Shamir secret sharing on a group of order q is defined as below:*

- $\{S_1, S_2, \dots, S_n\} \leftarrow \text{GrpShare}(S, n, t)$: Inputs a secret $S \in \mathbb{G}$, n and t , chooses a polynomial $f(x) = \sum_{i=1}^{t-1} a_i x^i$, where $a_i \xleftarrow{R} \mathbb{Z}_q$. Set $S_i = S \cdot g^{f(i)}$, $i \in [0, n]$. Obviously, $S_0 = S$.
- $S \leftarrow \text{GrpRecon}(Q)$: Inputs any t shares Q , reconstructs the secret $S \leftarrow S_0 = \prod_{i \in I} S_i^{\prod_{j \in I, j \neq i} \frac{-j}{i-j}}$, where I is the set containing indexes of the shares Q .

Shamir SS or Shamir SS on Group $\mathbb{G}$ should satisfy:

- **Correctness:** If the secret sharing process is correctly followed, any authorized set of participants can reconstruct the secret.
- **Secrecy:** Nothing information about the secret is revealed, given any unauthorized set of participants.

D. Linear Secret Sharing Scheme (LSSS)

Definition 4 (LSSS [46], [29], [24]). *Let $\mathcal{P}$ be a set of shareholders. A linear secret sharing scheme over a ring $\mathbb{Z}_q$ for an access structure $\mathbb{A}$ is a tuple (M, ρ) , where $M \in \mathbb{Z}_q^{|\mathbb{A}| \times l}$ is a share-generating matrix with $|\mathbb{A}|$ rows and l columns ² and ρ is a row-labeling function. The scheme consists of the following two algorithms:*

²We can consider $\mathbb{A}$ as an access control tree, as illustrated in Figure 1 and Figure 2, where $|\mathbb{A}|$ denotes the number of leaf nodes—or equivalently, the number of secret shares. The value of l , however, depends on the specific parsing algorithm introduced in [24], [69].

- $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{LSSSShare}(s, \mathbb{A})$: To share a value $s \in \mathbb{Z}_q$ under access structure $\mathbb{A}$, randomly choose $v_2, \dots, v_{|\mathbb{A}|} \xleftarrow{R} \mathbb{Z}_q$ and set $\vec{v} = [s, v_2, \dots, v_{|\mathbb{A}|}]^\top$. Then, $[s_1, \dots, s_{|\mathbb{A}|}] \leftarrow M\vec{v}$ is the vector of shares, where $s_i \in \mathbb{Z}_q$ belongs to shareholder $\rho(i)$ for each $i \in [1, \dots, |\mathbb{A}|]$.
 - $s \leftarrow \text{LSSSRecon}(\mathbb{A}, Q)$: Let $Q = \{s_i\}_{I_Q}$ be an authorized set for $\mathbb{A}$, where $I_Q = \{i | \rho(i) \in Q\}$ is the row indices in matrix M corresponding to $\mathbb{A}$. Let $M_Q \in \mathbb{Z}_N^{|I_Q| \times l}$ be the matrix formed by taking the subset of rows in M that are indexed by I_Q . Only if Q is an authorized set, there exists a vector $\vec{w}_Q \in \mathbb{Z}_N^{|I_Q|}$ such that $\vec{w}_Q^\top M_Q = [1, 0, \dots, 0]$. (Note that $\vec{w}_Q$ can be calculated using Gaussian elimination algorithm.) Further, the secret s can be reconstructed by $\sum_{i \in I_Q} \vec{w}_i s_i$.
- Similar to Shamir SS, LSSS should also satisfy **correctness** and **secrecy**.

E. Sigma Protocol and NIZK Proof

Let $\mathcal{R}$ be a relation, and define the corresponding language as $L_{\mathcal{R}} = \{x \mid \exists w : (x, w) \in \mathcal{R}\}$. The elements x and w are typically referred to as the statement and the witness, respectively. A Sigma protocol (P_1, P_2, V) is a three-move protocol between a prover and a verifier [42]. Firstly, the prover calculates a commitment $a = P_1(x)$; Secondly, the verifier randomly selects a challenge value $e \leftarrow \{0, 1\}^\lambda$, where λ is the security parameter; Lastly, the prover returns the response value $z = P_2(x, a, e)$. If $V(x, a, e, z) = 1$, the verifier is possesses that the prover possesses a witness w of the statement x such that $(x, w) \in \mathcal{R}$.

A non-interactive zero-knowledge (NIZK) proof of knowledge about w can be obtained by applying the Fiat-Shamir heuristic [43], with a random oracle. Model the random oracle using a random function, $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$. Consequently, the NIZK proof is denoted as $(a, e = H(x, a), z)$.

The NIZK proof has the properties of completeness, soundness and zero-knowledge. Completeness means that a valid proof always convinces the verifier; Soundness means that false statements cannot produce valid proofs (except negligibly); Zero-Knowledge means that the proof reveals nothing beyond the statement's truth.

IV. GENERALIZED SECRET SHARING (GSS)

We first define two types of GSS schemes, which differ based on the shared secret, $s \in \mathbb{Z}_q$ or $S \in \mathbb{G}$. Then, we initialize each type using two approaches, i.e., Shamir SS-based and LSSS-based. Technically, the Shamir SS-based primitive requires polynomial evaluation in the secret sharing and Lagrange interpolation in the secret recovery; The LSSS-based primitive needs to handle matrix multiplication both in secret sharing and recovery.

A. Definition

Definition 5 (GSS). A generalized secret sharing (GSS) scheme [18] allows a dealer to share a secret $s \in \mathbb{Z}_q$ using arbitrary monotone access structure. The secret s can be recovered only given an authorized set. A GSS scheme is defined by the following two algorithms.

- $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{GSSShare}(s, \mathbb{A})$: Inputs a value $s \in \mathbb{Z}_q$ and an access structure $\mathbb{A}$, outputs the secret shares $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\}$, where $|\mathbb{A}|$ is the number of leaf nodes under access tree $\mathbb{A}$ —or equivalently, the rows of the matrix M after transforming $\mathbb{A}$ to LSSS matrix M and row-labeling algorithm ρ .
- $s \leftarrow \text{GSSRecon}(\mathbb{A}, Q)$: Inputs an access structure $\mathbb{A}$ and an authorized set $Q \subseteq \{s_1, s_2, \dots, s_{|\mathbb{A}|}\}$, reconstructs the secret s .

Definition 6 (GSS on Group $\mathbb{G}$). Here we consider how to share secret $S \in \mathbb{G}$ and the dealer does not know the value s such that $S = g^s$. The GSS on group $\mathbb{G}$ of order q is defined as below:

- $\{S_1, S_2, \dots, S_{|\mathbb{A}|}\} \leftarrow \text{GrpGSSShare}(S, \mathbb{A})$: Inputs a value $S \in \mathbb{G}$ and an access structure $\mathbb{A}$, outputs the secret shares $\{S_1, S_2, \dots, S_{|\mathbb{A}|}\}$, where $|\mathbb{A}|$ is the same as in Definition 5.
- $S \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q)$: Inputs an access structure $\mathbb{A}$ and an authorized set $Q \subseteq \{S_1, S_2, \dots, S_{|\mathbb{A}|}\}$, reconstructs the secret S .

Similar to Shamir SS, GSS and GSS on Group $\mathbb{G}$ should also satisfy **correctness** and **secrecy**.

B. Shamir SS-based GSS

(1) **GSS based on Shamir SS:** Benaloh et al.[18] had introduced the construction, which can be obtained by replacing GrpShare (and GrpRecon) with Share (and Recon) in Figure 3. We omit the security proofs of **correctness** and **secrecy**.

(2) **GSS on Group $\mathbb{G}$ based on Shamir SS:** Without loss of generalization, an access structure $\mathbb{A}$ is a set of shareholders who are constraint by the threshold gate (refer to Figure 2 as an example). The non-leaf nodes are threshold gate values and the leaf nodes are identifiers of shareholders. The GSS scheme attaches a share for each (non-leaf or leaf) node and it is described by Figure 3.

Functionality GSS on Group $\mathbb{G}$ using Shamir SS

 $\{S_1, S_2, \dots, S_{|\mathbb{A}|}\} \leftarrow \text{GrpGSSShare}(S, \mathbb{A}) :$

set $F_R \leftarrow S$ for the root node R .

For non-leaf node x with n_x children nodes and threshold-gate t_x :

$\{S_{x_1}, S_{x_2}, \dots, S_{x_{n_x}}\} \leftarrow \text{GrpShare}(F_x, n_x, t_x)$

set $F_z \leftarrow S_{x_j}$, where z is the j th child of x

For the i th leaf node z in $\mathbb{A}$:

$S_i = S_{x_j}$, where z is the j th child of its parent node x

 $(S) \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q) :$

Denote *path* as the satisfied path (cf. Corollary 3)

For each node x in *path* from leaf to root:

$F_x \leftarrow \text{GrpRecon}(Q_x)$, where $Q_x \subseteq \{F_{x_1}, \dots, F_{x_{n_x}}\}$ generated at node x and $|Q_x| \geq t_x$. (**Remark:** if x is a leaf node, $F_x = Q_x$.)

$S \leftarrow F_R$, where R is the root node.

Fig. 3: GSS scheme on Group $\mathbb{G}$ based on Shamir SS

For any node in GSS scheme on $\mathbb{G}$, the **correctness** and **secrecy** can be derived directly from those of Shamir SS scheme on $\mathbb{G}$, which is given by Lemma 1. Thus, the **correctness** and **secrecy** of GSS on group $\mathbb{G}$ based on Shamir SS (Figure 3) are satisfied.

Lemma 1. *Shamir SS on group $\mathbb{G}$ (Definition 3) satisfies **correctness** and **secrecy**.*

Proof. **Correctness:** The shares are $S_i = S \cdot g^{f(i)}$, $i \in [0, n]$. Following the correctness of Shamir SS scheme, inputs a set of any t shares Q , we can reconstruct the secret $\prod_{i \in I} S_i^{\prod_{j \in I, j \neq i} \frac{-j}{i-j}} = S \cdot g^{f(0)} = S$, where I is the set containing indexes of the shares Q . **Secrecy:** If less than t shares can reconstruct the secret, then we have $\sum_{i \in I} f(i) \prod_{j \in I, j \neq i} \frac{-j}{i-j} = f(0)$, $|I| < t$, which violates the security properties of Shamir SS scheme. $\square$

C. LSSS-based GSS

(1) **GSS based on LSSS:** It is obvious that LSSS and GSS schemes have the same algorithms, including inputs and outputs. Figure 4 demonstrate that LSSS is an instance of GSS. Therefore, the **correctness** and **secrecy** are inherently guaranteed.

Functionality GSS using LSSS

 $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{GSSShare}(s, \mathbb{A}) :$

$\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{LSSSShare}(s, \mathbb{A})$

 $(s) \leftarrow \text{GSSRecon}(\mathbb{A}, Q) :$

$s \leftarrow \text{LSSSRecon}(\mathbb{A}, Q)$

Fig. 4: GSS based on LSSS

(2) **GSS on Group $\mathbb{G}$ using LSSS:** We first depict how to construct LSSS on group $\mathbb{G}$ by Figure 5. The construction resembles the LSSSShare and LSSSRecon algorithms of the LSSS scheme, as defined in Definition 4. We prove that it satisfies the **correctness** and **secrecy** by Lemma 2.

Lemma 2. *LSSS on group $\mathbb{G}$ (Figure 5) satisfies **correctness** and **secrecy**.*

Proof. **Correctness:** The shares are $S_i = S^{s_i} = g^{s \cdot s_i}$ for $i \in [1, \dots, |\mathbb{A}|]$. Q is a authorized set with $I_Q = \{i | \rho(i) \in Q\}$ and $M_Q \in \mathbb{Z}_N^{|I_Q| \times l}$, which means there exists $w_Q \in \mathbb{Z}_N^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \dots, 0]$. Following the correctness of LSSS scheme, we have $\sum_{i \in I_Q} w_i \cdot s_i = 1$, thus $S = \prod_{i \in I_Q} S_i^{w_i} = \prod_{i \in I_Q} g^{s \cdot s_i \cdot w_i} = (g^s)^{\sum_{i \in I_Q} w_i \cdot s_i} = g^s$. **Secrecy:** If unauthorized set Q can reconstruct $g^s = \prod_{i \in I_Q} S_i^{w_i}$, then it can find $w_Q \in \mathbb{Z}_N^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \dots, 0]$, which violates the security properties of LSSS scheme, as only authorized sets can find such w_Q . $\square$

Then, we apply the construction of Figure 5 to construct GSS on group $\mathbb{G}$, as Figure 6 presents. It can be observed that the GrpGSSShare and GrpGSSRecon algorithms directly invoke GrpLSSSShare and GrpLSSSRecon, respectively. Apparently, LSSS on group $\mathbb{G}$ can be regarded as an instance of GSS on group $\mathbb{G}$. Thus, **correctness** and **secrecy** are inherently guaranteed by Lemma 2.

Functionality LSSS on Group $\mathbb{G}$

$\{S_1, S_2, \dots, S_{|\mathbb{A}|}\} \leftarrow \text{GrpLSSSShare}(S, \mathbb{A})$: denote $S = g^s$ and s is unknown.

parse $\mathbb{A}$ as (M, ρ) leveraging the algorithms in [24] or [69]

$\vec{v} = [1, v_2, \dots, v_l]^\top$, where $v_2, \dots, v_l \xleftarrow{R} \mathbb{Z}_q$.

$(s_1, \dots, s_{|\mathbb{A}|}) \leftarrow M\vec{v}$

$S_i = S^{s_i} = g^{s \cdot s_i}$ for $i \in [1, \dots, |\mathbb{A}|]$

$(S) \leftarrow \text{GrpLSSSRecon}(\mathbb{A}, Q)$:

parse $\mathbb{A}$ as (M, ρ)

calculate I_Q as $I_Q = \{i | \rho(i) \in Q\}$

set $M_Q \in \mathbb{Z}_N^{|I_Q| \times l}$, where M_Q is the subset of rows in M that are indexed by I_Q

find $w_Q \in \mathbb{Z}_N^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \dots, 0]$

$S \leftarrow \prod_{i \in I_Q} S_i^{w_i}$

Fig. 5: LSSS on Group $\mathbb{G}$

Functionality GSS on Group $\mathbb{G}$ using LSSS

$\{S_1, S_2, \dots, S_{|\mathbb{A}|}\} \leftarrow \text{GrpGSSShare}(S, \mathbb{A})$:

$\{S_1, S_2, \dots, S_{|\mathbb{A}|}\} \leftarrow \text{GrpLSSSShare}(S, \mathbb{A})$

$(S) \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q)$:

$S \leftarrow \text{GrpLSSSRecon}(\mathbb{A}, Q)$

Fig. 6: GSS on Group $\mathbb{G}$ based on LSSS

V. PUBLICLY VERIFIABLE GENERALIZED SECRET SHARING (PVGSS)

A. Definition

Definition 7 (PVGSS). A Publicly verifiable generalized secret sharing (PVGSS) scheme allows a dealer to split a secret using an arbitrary monotone access structure in a publicly verifiable manner. Hence, shares are encrypted and can be delivered in a public communication channel. Besides, the correctness of the encrypted shares can be verified by any external party. The PVGSS scheme includes five algorithms.

- $(sk_i, pk_i) \leftarrow \text{PVGSSSetup}(\lambda)$: Inputs security parameter λ , outputs a key pair (sk_i, pk_i) for each shareholder P_i .
- $(\{C_i\}, \text{prf}_s) \leftarrow \text{PVGSSShare}(\mathbb{A}, \{pk_i\})$: Inputs an access structure $\mathbb{A}$ and the public keys $\{pk_i\}$, outputs the encrypted shares $\{C_i\}$ and the corresponding NIZK proofs prf_s for a secret s randomly chosen from $\mathbb{Z}_q$.
- $(0/1) \leftarrow \text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A})$: Inputs encrypted shares $\{C_i\}$, NIZK proofs prf_s and the access structure $\mathbb{A}$, outputs 1 if the shares are correctly encrypted. Otherwise, outputs 0.
- $(S_i, \text{prf}_{S_i}) \leftarrow \text{PVGSSPreRecon}(C_i, sk_i)$: Inputs ciphertext C_i and the corresponding secret key sk_i , outputs a decrypted share S_i .
- $(0/1) \leftarrow \text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, pk_i)$: Inputs the encrypted share C_i , the decrypted share S_i and the public key pk_i , outputs 1 if S_i is corrected decrypted from C_i .
- $(S) \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q)$: Inputs the access structure $\mathbb{A}$ and a set of decrypted shares Q , outputs the recovered secret $S(= g^s)$ given Q is authorized.

PVGSS scheme is required to achieve below security properties.

- **Correctness:** The scheme ensures that if the secret sharing process is correctly followed, any authorized subset of participants can reconstruct the secret S .
- **Verifiability:** Anyone can verify that the encrypted shares provided by the dealer are correct without knowing the actual secret. Besides, users can also check correctness of the decrypted shares in the reconstruction phase.
- **IND1-secrecy:** The secret remains hidden from unauthorized parties prior to the reconstruction phase.

Definition 8 (Correctness). The PVGSS is correct if for all authorized set Q of $\mathbb{A}$, each s in the secret space,

$$\Pr \left[\begin{array}{l} \text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}) = 1 \wedge \\ \text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, pk_i) = 1, \\ \forall i \in I_Q \wedge S' = g^s \end{array} \middle| \begin{array}{l} (sk_i, pk_i) \leftarrow \text{PVGSSSetup}(\lambda); \\ (\{C_i\}, \text{prf}_s) \leftarrow \text{PVGSSShare}(\mathbb{A}, \{pk_i\}) \\ \forall i \in I_Q, (S_i, \text{prf}_{S_i}) \leftarrow \text{PVGSSPreRecon}(C_i, sk_i) \\ (S') \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q); \end{array} \right] = 1$$

Definition 9 (Verifiability of Encrypted Shares). *The PVGSS satisfies verifiability of encrypted shares if for every PPT $\mathcal{A}$,*

$$\Pr \left[\text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}) = 1 \wedge \left| \{(\{pk_i\}, \{sk_i\}) \leftarrow \text{PVGSSSetup}(\lambda), (\{C_i\}, \text{prf}_s, \mathbb{A}) \leftarrow \mathcal{A}(\{pk_i\})\} \right| \leq \text{negl}(\lambda) \right]$$

Definition 10 (Verifiability of Share Decryption). *The PVGSS satisfies verifiability of share decryption if for every PPT $\mathcal{A}$,*

$$\Pr \left[\text{PVGSSKeyVrf}(C, S, \text{prf}_S, pk) = 1 \wedge \left| \{(\{pk_i\}, \{sk_i\}) \leftarrow \text{PVGSSSetup}(\lambda), (\{sk\} \in \{sk_i\} \text{ s.t. } (S, \text{prf}_S) \leftarrow \text{PVGSSPreRecon}(C, sk)) \mid (pk, C, S) \leftarrow \mathcal{A}(\{pk_i\}), pk \in \{pk_i\}\} \right| \leq \text{negl}(\lambda) \right]$$

Definition 11 (IND1-secrecy). *IND1-secrecy standards for indistinguishability of secrets. PVGSS achieves IND1-secrecy if for any polynomial time adversary $\mathcal{A}$ without an authorized set, $\mathcal{A}$ win the following game played against a challenger with negligible advantage.*

- 1) The challenger initiates the PVGSSSetup algorithm for all honest parties and publishes the public keys $\{pk_i\}$.
- 2) $\mathcal{A}$ generates a challenge access structure $\mathbb{A}^*$. $\mathcal{A}$ chooses a maximum set of indexes $U \subset [1, \dots, |\mathbb{A}^*|]$, where the shares corresponding to U do not satisfy the access structure $\mathbb{A}^*$. It sends $\mathbb{A}^*, U$ to the challenger.
- 3) The challenger randomly selects values x_0 and x_1 from the secret space. It then randomly selects $b \in \{0, 1\}$. Further, it invokes $(\{C_i^*\}, \text{prf}_{x_0}) \leftarrow \text{PVGSSShare}(\mathbb{A}^*, \{pk_i\})$, which indicates that x_0 is used as the secret. The challenger sends $(\{C_i^*\}, \text{prf}_{x_0}, x_b)$ to $\mathcal{A}$.
- 4) $\mathcal{A}$ then outputs a guess $b' \in \{0, 1\}$.

The $\mathcal{A}$'s advantage in the game is defined as $|\Pr[b = b'] - \frac{1}{2}|$.

B. PVGSS Construction Based on GSS

Usually, PVSS schemes are constructed based on SS. Informally speaking, PVSS combines SS and public key encryption with verifiability. Intuitively, PVGSS can be constructed similarly by replacing SS with GSS. We leverage the original GSS scheme in the sharing phase and employ GSS on group $\mathbb{G}$ in the reconstruction phase. Figure 7 illustrate our mindset of constructions vividly.

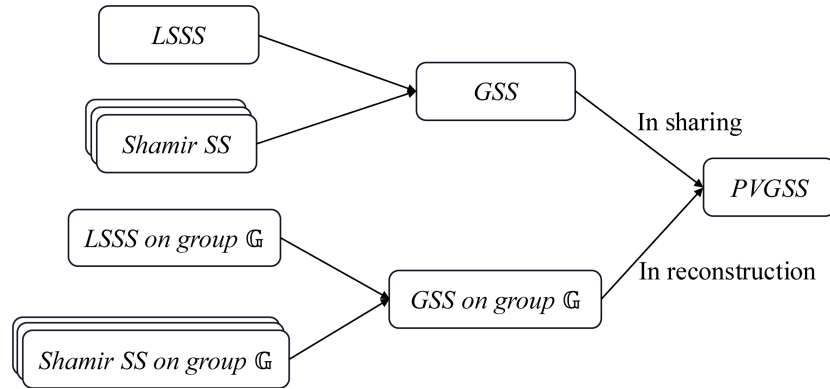


Fig. 7: High-level overview of our PVGSS constructions

Figure 8 demonstrates the general PVGSS construction by taking advantage of GSS scheme and NIZK proofs. The NIZK proofs are acquired by incorporating the Sigma protocol [42] and the Fiat-Shamir heuristic [43]. The initiation phase comprises the PVGSSSetup algorithm. The sharing phase involves the PVGSSShare and PVGSSVerify algorithms. The reconstruction phase includes PVGSSPreRecon, PVGSSKeyVrf, and GSSRecon algorithms.

In the PVGSSSetup algorithm, each shareholder j generates its key pair (sk_i, pk_i) , where $pk_i = g^{sk_i}$ and sk_i is randomly chosen from $\mathbb{Z}_q$.

In the PVGSSShare algorithm, the dealer first invokes GSSShare algorithm for secret $s \in \mathbb{Z}_q$ to generate secret shares based on the access structure $\mathbb{A}$. Note that a PVGSS is not merely a combination of multiple PVSS instances, since it outputs encrypted shares only for the leaf nodes. As previously introduced, $|\mathbb{A}|$ denotes the number of leaf nodes in the access structure $\mathbb{A}$. Then, it hides the $|\mathbb{A}|$ shares directly using the targeted shareholder's public key, i.e., $pk_i^{s_i}$. Next, it executes the Sigma protocol and Fiat-Shamir heuristic to generate NIZK proofs (cf. Section III-E). Specifically, $\{C'_i\}, c, \{\hat{s}_i\}$ are the commitment, the challenge value and the response value, respectively. Therefore, the NIZK proof prf_s is $(\{C'_i\}, c, \{\hat{s}_i\})$. To this end, the dealer has successfully generated encrypted secret shares $\{C_i\}$ and the corresponding NIZK proof prf_s which can be adopted to prove the dealer's honesty. Different from the LDEI protocol in Albatross [48], which directly outputs the polynomial $\hat{p}(X)$ as part of the Sigma protocol response values, PVGSS transmits $|\mathbb{A}|$ polynomial evaluation points $\{\hat{s}_i\}$ of $\hat{p}(X)$.

In the PVGSSVerify algorithm, any (external) party can check whether the dealer has honestly performed the sharing phase. Due to the presence of $|\mathbb{A}|$ independent shares $\{s_i\}$, the verifier needs to perform verification using two equations. The first equation (executed $|\mathbb{A}|$ times) is equivalent to the Schnorr signature [34], enabling the dealer to prove knowledge of each individual share s_i . The second equation implies that all the response values $\{\hat{s}_i\}$ are correctly generated using the challenge value c . The second equation allows the dealer to prove that the shares embedded in $\{C_i\}$ are bound to the secret s and $S = g^s$. Refer to Theorem 1 for details about the correctness.

In the PVGSSPreRecon algorithm, the shareholder, holding sk_i , decrypts C_i with its secret key sk_i and obtains $S_i = g^{s_i}$. Additionally, the shareholder employs the discrete logarithm equality (DLEQ) algorithm [70] to generate a non-interactive zero-knowledge (NIZK) proof prf_{S_i} , demonstrating that C_i and pk_i share the same exponent sk_i .

The PVGSSKeyVrf algorithm directly invokes the corresponding DLEQ verification algorithm, $DLEQVrf$, to verify the correctness of S_i provided by the shareholder. Note that no information about sk_i is leaked during this process.

Given enough authorized decrypted shares, i.e., an authorized set Q , the PVGSSRecon algorithm can reconstruct a secret S using GrpGSSRecon algorithm. Note that GrpGSSRecon, rather than GSSRecon, is used to recover the secret, since in our PVGSS scheme the secret S resides in $\mathbb{G}$, consistent with many prior PVSS constructions [13], [48].

Functionality PVGSS Based on GSS

$(sk_i, pk_i) \leftarrow \text{PVGSSSetup}(\lambda) :$

$$sk_i \xleftarrow{R} \mathbb{Z}_q$$

$$pk_i \leftarrow g^{sk_i}$$

$(\{C_i\}, \text{prf}_s) \leftarrow \text{PVGSSShare}(\mathbb{A}, \{pk_i\}) :$

$$s \xleftarrow{R} \mathbb{Z}_q$$

$$\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{GSSShare}(s, \mathbb{A})$$

$$C_i = pk_i^{s_i} \quad \forall i \in [1, \dots, |\mathbb{A}|]$$

$$s' \xleftarrow{R} \mathbb{Z}_q$$

$$(s'_1, \dots, s'_{|\mathbb{A}|}) = \text{GSSShare}(s', \mathbb{A})$$

$$C'_i = pk_i^{s'_i} \quad \forall i \in [1, \dots, |\mathbb{A}|]$$

$$c \leftarrow H(\{C_i\}, \{C'_i\})$$

$$\hat{s} \leftarrow s' - c \cdot s$$

$$\hat{s}_i \leftarrow s'_i - c \cdot s_i \quad \forall i \in [1, \dots, |\mathbb{A}|]$$

$$\text{prf}_s \leftarrow (\{C'_i\}, c, \hat{s}, \{\hat{s}_i\})$$

$(0/1) \leftarrow \text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}) :$

$$C'_i \stackrel{?}{=} C_i^c \cdot pk_i^{\hat{s}_i} \quad \forall i \in [1, \dots, |\mathbb{A}|]$$

$$\hat{s} \stackrel{?}{=} \text{GSSRecon}(\mathbb{A}, \{\hat{s}_i\}).$$

$(S_i, \text{prf}_{S_i}) \leftarrow \text{PVGSSPreRecon}(C_i, sk_i) :$

$$S_i = C_i^{1/sk_i} = g^{s_i}$$

$$\text{prf}_{S_i} = DLEQ(S_i, C_i, g, pk_i, s_i)$$

$(0/1) \leftarrow \text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, pk_i) :$

$$\text{true} \stackrel{?}{=} DLEQVrf(S_i, C_i, g, pk_i, \text{prf}_{S_i})$$

$(S) \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q) :$

$$S \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q) \text{ where } S = g^s \text{ and } Q \subseteq \{S_1, \dots, S_{|\mathbb{A}|}\} \text{ is an authorized set of } \mathbb{A}.$$

Fig. 8: PVGSS scheme based on GSS and NIZK proofs

C. Complexity Analysis

Recall that: 1) PVGSS schemes output encrypted shares only for the leaf nodes defined by the access structure $\mathbb{A}$; 2) $|\mathbb{A}|$ denotes the number of leaf nodes in $\mathbb{A}$; and 3) Q represents an authorized set under $\mathbb{A}$.

Computational Complexity: The computational costs of matrix operations (including generation and multiplication in the LSSS-based construction) and polynomial operations (including evaluation and Lagrange interpolation in the Shamir SS-based construction), are negligible compared to group exponentiation in $\mathbb{G}$. As such, these operations are excluded from the complexity analysis.

In the sharing phase, the PVGSSShare algorithm costs $2|\mathbb{A}|$ group exponentiations, where each C_i and C'_i requires an exponentiation. The PVGSSVerify algorithm requires $2|\mathbb{A}|$ exponentiations to verify the correctness of each C_i . Therefore,

the distribution complexity is $O(|\mathbb{A}|)$. In the reconstruction phase, the PVGSSPreRecon algorithm costs 3 exponentiations for each shareholder, including the calculation of prf_{s_i} from DLEQ. The PVGSSKeyVrf algorithm requires 4 exponentiations as required by *DLEQVrf*. The PVGSSRecon algorithm takes $|Q|$ exponentiations, where Q denotes the authorized set for the access structure $\mathbb{A}$. Thus, the reconstruction complexity is $O(|Q|)$.

Communicational Complexity: In the PVGSS sharing phase, the dealer outputs $(\{C_i\}, \text{prf}_s)$, which consists of $2|\mathbb{A}|$ $\mathbb{G}$ elements and $(|\mathbb{A}| + 2) \mathbb{Z}_q$ elements. During the reconstruction phase, each user needs to collect $(3\mathbb{G} + 2\mathbb{Z}_q)|Q|$ elements, as each correct prf_{s_i} contains 2 elements on $\mathbb{G}$ and 2 elements on $\mathbb{Z}_q$. Therefore, the communication complexity is $O(|\mathbb{A}|)$ during the sharing phase and $O(|Q|)$ during the reconstruction phase, respectively.

D. Comparison with Albatross PVSS [48]

The PVGSS constructions shares some similarities with the Albatross PVSS scheme, particularly in leveraging the Sigma protocol and Fiat-Shamir heuristic to achieve public verifiability. However, there are key differences.

- 1) PVGSS research is orthogonal to PVSS, as it targets arbitrary monotone access structures while maintaining the flexibility to reduce to a PVSS scheme akin to Albatross [48]. Moreover, PVGSS is not a straightforward aggregation of multiple PVSS instances, making it a non-trivial primitive.
- 2) The LDEI protocol in Albatross, which proves knowledge of a polynomial $p(X)$, leverages the polynomial $\hat{p}(X)$ as part of the response values. In contrast, our PVGSS scheme transmits $|\mathbb{A}|$ polynomial evaluation points $\{\hat{s}_i\}$ of $\hat{p}(X)$.
- 3) Though $\{\hat{s}_i\}$ is essentially equal to $\hat{p}(X)$, it is a different representation. $\hat{p}(x)$ directly reveals the coefficients and degree of the polynomial, while $\{\hat{s}_i\}$ needs to be “interpolated” to obtain the polynomial coefficients or degree. Thus, our scheme requires an additional verification equation (i.e., $\hat{s} = \text{GSSRecon}(\mathbb{A}, \{\hat{s}_i\})$), which emphasizes the proof of knowledge of the secret s ($= p(0)$), rather than the whole polynomial $p(X)$. The verification algorithm is similar to the application of Reed-Solomon code in [13] or SCRAPE test in [47], but it is general for arbitrary access structure.
- 4) If we consider applying the Albatross approach to another PVGSS implementation, recursive response polynomials (including those generated at non-leaf nodes) would need to be generated. The question of whether—and how—these polynomials can be effectively represented and implemented for building PVGSS is of independent interest.
- 5) The verification algorithm is unified through the application of GSSRecon algorithm for both Shamir SS-based and LSSS-based PVGSS. The GSSRecon algorithm functions without the need for shares at non-leaf nodes in an access structure, thus serving as a core component of the PVGSS scheme.
- 6) Suppose we downgrade our PVGSS scheme to a PVSS setting and compare it with Albatross. The downgraded PVSS and Albatross schemes share a nearly identical sharing phase but differ slightly in their verification algorithms. For example, in the LSSS-based PVGSS scheme, the PVGSSVerify procedure depends on the GSSRecon algorithm—specifically LSSSRecon—to recover $\hat{s}$, a process that involves matrix computation. In contrast, Albatross’s LDEI verification algorithm requires n point evaluations of the polynomial $\hat{p}(X)$. A detailed comparison of their computational costs may be of independent interest.

E. Security Analysis

Theorem 1 (Correctness). *The PVGSS construction satisfies correctness defined by Definition 8.*

Proof. If the dealer is honest and outputs $\{C_i\}$ and prf_s using GSSShare based on the access structure $\mathbb{A}$, then $\text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}) = 1$. That is implied by Schnorr signature [34], where each $(C_i, c, (C'_i, \hat{s}))$ contained in prf_s comprise of a Sigma protocol [34] and the GSSRecon algorithm. The GSSRecon algorithm can successfully recover $\hat{s}$ due to the additive homomorphism of Shamir SS and LSSS schemes. If PVGSSPreRecon is applied to (C_i, sk_i) honestly, then $S_i = g^{s_i}$ and $\text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, \text{pk}_i) = 1, \forall i \in I_Q$. Given an authorized set Q of $\mathbb{A}$, we have the reconstructed value $S' \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q)$ which in turn equals $S = g^s$, due to **correctness** of GSS on group $\mathbb{G}$. The correctness has been proved in Section IV-B and Section IV-C. Therefore, the PVGSS satisfies correctness defined by Definition 8. $\square$

Theorem 2 (Verifiability of Encrypted Shares). *If NIZK proof with soundness error negligible in λ , then PVGSS has verifiability of encrypted shares, defined by Definition 9.*

Proof. If $\text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}) = 1$, then except with NIZK soundness error $\text{negl}(\lambda)$, there exists s_i such that $C_i = \text{pk}_i^{s_i}$ and $C'_i = C_i^c \cdot \text{pk}_i^{s_i}$. Further, $\hat{s} = \text{GSSRecon}(\mathbb{A}, \{\hat{s}_i\})$ holds. If $\{C_i\}$ are not valid shares of a PVGSS secret, then $\text{GSSRecon}(\mathbb{A}, \{s_i\})$ is denoted as $\perp$. Further, $\text{GSSRecon}(s'_i) = \text{GSSRecon}(\mathbb{A}, \{\hat{s}_i + c \cdot s_i\}) = \text{GSSRecon}(\mathbb{A}, \{\hat{s}_i\}) + \text{GSSRecon}(\mathbb{A}, \{c \cdot s_i\}) = \hat{s} + c \cdot \perp$. It means that $\{s'_i\}$ are not valid shares of a secret. Therefore, no valid s' and s exists to generate $\hat{s}$, violating $\hat{s}$ as response value of Sigma protocol with NIZK soundness error $\text{negl}(\lambda)$. Therefore, the PVGSS satisfies verifiability of encrypted shares by Definition 9. $\square$

Theorem 3 (Verifiability of Share Decryption). *If a shareholder communicates a decryption share $\hat{S}_i$ in the reconstruction phase, then it is detected with probability 1, defined by Definition 10.*

Proof. The verifiability of share decryption is evident from the properties of the DLEQ algorithm [70], where sk_i serves as the common exponent in both C_i and pk_i .

Theorem 4 (IND1-secrecy). *The PVGSS is IND1-secrecy, defined by game Definition 11, under DDH assumption.*

Proof. Firstly, the NIZK proofs prf_s of PVGSSShare reveal nothing about the secret s or shares $\{s_i\}$, due to zero-knowledge property. Then, we argue that, if there exists an adversary $\mathcal{A}_{IND}$ which can break the **IND1-secrecy** property of PVGSS, then there exists an adversary $\mathcal{A}_{DDH}$ which can use $\mathcal{A}_{IND}$ to break DDH assumption.

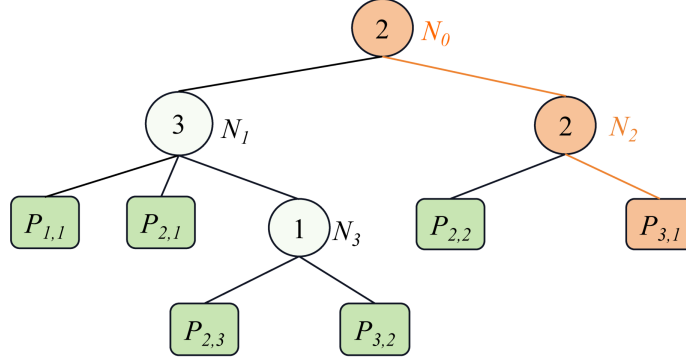


Fig. 9: A maximum unauthorized set is shown in green, while an unsatisfied path is highlighted in orange.

Shamir SS-based PVGSS: The security game requires $\mathcal{A}_{IND}$ controls a unauthorized set. Thus, there exists a path in which all nodes are not satisfied by the unauthorized set, as stated in Corollary 3. Taking Figure 9 as an example. $\mathcal{A}_{IND}$ can query a maximum set of decrypted shares for nodes: $\{P_{1,*}, P_{2,*}, P_{3,2}\}$.³ Thus, nodes $(N_0, N_2, P_{3,1})$ compose an unsatisfied path. From leaf to root in the unsatisfied path, we assume each node is not satisfied by a maximum unauthorized set. That means, we only need to consider a (t, n) threshold access structure $\mathbb{A}^*$, and assume that $\mathcal{A}_{IND}$ corrupts the $t - 1$ first shareholders.

Let $(h, h^\alpha, h^\beta, h^\gamma)$ be an instance of the DDH problem. Then, $\mathcal{A}_{DDH}$ using $\mathcal{A}_{IND}$ can simulate an security game as follows:

- 1) $\mathcal{A}_{IND}$ generates a (t, n) threshold access structure $\mathbb{A}^*$ and sends it to the challenger. $\mathcal{A}_{IND}$ sets the maximum unauthorized set index $U = [1, \dots, t - 1]$. $\mathcal{A}_{IND}$ sends $(\mathbb{A}^*, U)$ to the $\mathcal{A}_{IND}$.
- 2) The challenger sets $g = h^\alpha$. For $t \leq i \leq n$, $\mathcal{A}_{DDH}$ randomly choose $r_i \xleftarrow{R} \mathbb{Z}_q$ and sends the values $pk_i = h^{r_i}$ to $\mathcal{A}_{IND}$. This implicitly defines $sk_i = r_i/\alpha$ for $i \in [t, n]$, though α is unknown to the challenger.
- 3) For $1 \leq i \leq t - 1$, $\mathcal{A}_{IND}$ randomly chooses sk_i and sets $pk_i = g^{sk_i}$, then, it sends these to the challenger.
- 4) For $1 \leq i \leq t - 1$, the challenger samples $s_i \xleftarrow{R} \mathbb{Z}_q$ and sets $C_i^* = pk_i^{s_i}$.

For $1 \leq i \leq t - 1$, the challenger randomly chooses s_i from $\mathbb{Z}_q$ and calculates h^{s_i} . Then, given t points, i.e., $\{h^{s_i}\}$ and h^β , $\mathcal{A}_{DDH}$ can use Lagrange interpolation to compute $S_i^* = h^{s_i} = h^{p(i)}$ for $t \leq i \leq n$, where $p(\cdot)$ is the interpolated polynomial with degree $t - 1$ such that $h^{p(0)} = h^\beta$. Further, it also generates shares $C_i^* = (S_i^*)^{r_i} = pk_i^{s_i}$.

To simulate the NIZK proof prf_β , $\mathcal{A}_{DDH}$ randomly chooses a $(t - 1)$ -degree polynomial $\hat{p}(x)$ and $c \in \mathbb{Z}_q$. Then, $\mathcal{A}_{DDH}$ calculates $\hat{s}_i = \hat{p}(i)$ and sets $C_i'^* = (C_i^*)^c \cdot pk_i^{\hat{s}_i}$ for $1 \leq i \leq n$. Note that the random oracle $H(\cdot)$ is programmed so that $c \equiv H(\{C_i^*\}, \{C_i'^*\})$. Apparently, the simulated tuple $(\{C_i'^*\}, c, \{\hat{s}_i\})$ forms a valid NIZK proof prf_β for the sharing phase.

Finally, it sends $\{C_i^*\}$, prf_β and h^γ to $\mathcal{A}_{IND}$, where h^γ plays the role of x_b in the IND1-secrecy game.

- 5) Output: $\mathcal{A}_{IND}$ make a guess b' .

If $b' = 0$, $\mathcal{A}_{DDH}$ guess that $\gamma = \alpha \cdot \beta$, if $b' = 1$, $\mathcal{A}_{DDH}$ guess that γ is a random element. The information that $\mathcal{A}_{IND}$ receives in step 4), is distributed exactly like a sharing of the value $g^\beta = h^{\alpha \cdot \beta}$. If h^γ sent to $\mathcal{A}_{IND}$ is the secret shared by PVGSS, $\gamma = \alpha \cdot \beta$. So the advantage of $\mathcal{A}_{DDH}$ is the same as the advantage of $\mathcal{A}_{IND}$.

LSSS-based PVGSS: Consider $\mathbb{A}^*$ is a LSSS access structure (M^*, ρ) , each row corresponds to one shareholder. Assume $\mathcal{A}_{IND}$ can corrupts a max unauthorized set U of $\mathbb{A}^*$. Let $I_U = \{i | \rho(i) \in U\}$ be the rows by the row indexes U . To maximize the attacking capacity of $\mathcal{A}_{IND}$, the set I_U may include partial rows for some party $\rho(i)$. Taking Figure 9 as an example, the row corresponding to leaf node $P_{3,2}$ is included in I_U , while the row corresponding to $P_{3,1}$ is not. M_i denotes the i th row of M and $M_{i,j}$ denotes the i th row and j th column of M .

Let $(h, h^\alpha, h^\beta, h^\gamma)$ be an instance of the DDH problem. Then $\mathcal{A}_{DDH}$ using $\mathcal{A}_{IND}$ can simulate an IND game as follows:

- 1) $\mathcal{A}_{IND}$ generates an generalized access structure $\mathbb{A}^* = (M^*, \rho)$ and sends (M^*, ρ, I_U) to the challenger.
- 2) The challenger sets $g = h^\alpha$. For $i \notin I_U$, $\mathcal{A}_{DDH}$ randomly chooses $r_i \xleftarrow{R} \mathbb{Z}_q$ and sends the values $pk_i = h^{r_i}$ to $\mathcal{A}_{IND}$. This implicitly defines $sk_i = r_i/\alpha$ for $i \notin I_U$, though α is unknown to the challenger.

³The maximum authorized set is not unique.

- 3) For $i \in I_U$, $\mathcal{A}_{IND}$ randomly chooses sk_i and sets $pk_i = g^{sk_i}$, then, it sends these to the challenger.
- 4) Given M^* , there exists a vector $\vec{w} = [w_1, \dots, w_{|\mathbb{A}^*|}] \in \mathbb{Z}_N^{|\mathbb{A}^*|}$ such that $\vec{w}^\top M^* = [1, 0, \dots, 0]$. For $j \in I_U$, the challenger samples $s_j \xleftarrow{R} \mathbb{Z}_q$ and sets $C_j^* = pk_j^{s_j}$. Since I_U is a max unauthorized set index of $\mathbb{A}^*$, given any additional row $i \notin I_U$, the access structure will be satisfied and the secret h^β can be recovered. Therefore, $\mathcal{A}_{DDH}$ computes $S_i^* = h^{s_i} = (\frac{h^\beta}{h^{\sum_{j \in I_U} w_j s_j}})^{\frac{1}{w_i}}$ for all $i \notin I_U$ and then it generates shares $C_i^* = (S_i^*)^{r_i} = pk_i^{s_i}$. To simulate the NIZK proof prf_β , $\mathcal{A}_{DDH}$ randomly chooses $\hat{s} \in \mathbb{Z}_q$ and $c \in \mathbb{Z}_q$. Then, $\mathcal{A}_{DDH}$ computes shares $\{\hat{s}_i\}$ for secret $\hat{s}$ under the access structure $\mathbb{A}^*$ and sets $C_i'^* = (C_i^*)^c \cdot pk_i^{\hat{s}_i}$ for all i . Note that the random oracle $H(\cdot)$ is programmed so that $c \equiv H(\{C_i^*\}, \{C_i'^*\})$. Apparently, the simulated tuple $(\{C_i'^*\}, c, \hat{s}, \{\hat{s}_i\})$ forms a valid NIZK proof prf_β for the sharing phase. Finally, the challenger sends $(\{C_i^*\}, \text{prf}_\beta)$ and h^γ to $\mathcal{A}_{IND}$, where h^γ plays the role of x_b in the ND1-secrecy game.
- 5) Output: $\mathcal{A}_{IND}$ make a guess b' .
If $b' = 0$, $\mathcal{A}_{DDH}$ guess that $\gamma = \alpha \cdot \beta$, if $b' = 1$, $\mathcal{A}_{DDH}$ guess that γ is a random element. The information that $\mathcal{A}_{IND}$ receives in step 4), is distributed exactly like a sharing of the value $g^\beta = h^{\alpha \cdot \beta}$. If h^γ sent to $\mathcal{A}_{IND}$ is the secret shared by PVGSS, $\gamma = \alpha \cdot \beta$. So the advantage of $\mathcal{A}_{DDH}$ is the same as the advantage of $\mathcal{A}_{IND}$.

□

VI. DEX BASED ON PVGSS

While centralized exchanges (CEXs), such as Binance⁴ and Coinbase⁵, offer high liquidity, advanced features, and ease of use, they also come with inherent risks. These include reliance on a central authority, which may lead to vulnerabilities such as security breaches, mismanagement, or censorship. Additionally, users must relinquish control of their private keys, creating potential single points of failure (SPOF). To address these challenges, Decentralized exchanges (DEXs) have emerged as an alternative, providing a trustless and transparent trading environment where users retain full control over their assets and transactions are secured through blockchain technology. DEX inherently addresses the fair exchange problem, which has been proven impossible without a trusted third party [72].

Hash time-locked contract (HTLC)⁶ and adaptor signature [52], [55] have shown promising opportunity in implementing fair atomic swap [53], [54], which is the primary requirement for designing DEXs. However, these methods have flaws in fostering the DeFi markets, limiting their world-wide application in DEX. There is no straightforward arbitration mechanism, leading to long time waits or risks in the event of disputes or non-cooperation, such as a party refusing to reveal a preimage or signature. Besides, these methods focus primarily on ensuring fairness between the two trading parties through cryptographic mechanisms. [56] They often overlook the broader economic incentives for other participants, such as liquidity providers or arbitrageurs, who play a crucial role in fostering a thriving market ecosystem.

Automated market maker (AMM) based DEXs, such as Uniswap⁷, eliminate the need for direct counterparties by using liquidity pools and mathematical algorithms to determine token prices [57]. This approach simplifies trading, improves accessibility, and removes the reliance on strict time-locks or preimage disclosures, making it an attractive option for both new and experienced users. Nevertheless, liquidity providers (LPs) have to store token pairs into DEX and face impermanent loss when asset prices in the pool change significantly. High slippage occurs during large trades, making AMMs less efficient for high-value transactions. Vulnerability to front-running and miner extractable value attacks exposes traders to unfair costs [58], [59]. AMMs also suffer from capital inefficiency, requiring large liquidity to reduce slippage, while locking funds with suboptimal returns [60].

Due to the existing obstacles, we propose a novel DEX built on a transparent and trustless protocol using the proposed PVGSS scheme. Trades are executed on smart contract without manipulation, ensuring equitable outcomes and preventing exploitation by any participant. Our design incentivizes arbitrageurs (watchers) by rewarding them for providing passive arbitration services, ensuring active participation and fostering a robust and dynamic marketplace. Unlike AMM-based models, the arbitrageurs in our DEX has no impermanent loss and traders exchange at their expected prices without slippage. Table II compares the above mentioned exchanges.

A. High-Level Overview

Based on the proposed PVGSS scheme, we design a DEX to allow exchangers to swap tokens fairly and simultaneously. In this research, we merely focus on ERC-20⁸ token exchange. The DEX involves two roles: exchangers and watchers. Anyone can be exchangers and watchers. Each exchanger holds specific types of ERC-20 tokens, while multiple watchers collectively form a passive notary committee to address potential disputes.

⁴<https://www.binance.com/en>

⁵<https://www.coinbase.com>

⁶https://en.bitcoin.it/wiki/Hash_Time_Locked_Contracts

⁷<https://app.uniswap.org>

⁸<https://eips.ethereum.org/EIPS/eip-20>

TABLE II: Comparisons of different exchanges

Exchanges	Fariness provider	Arbitrageurs	No SPOF	Arbitration	No slippage	Divergence Losslessness
CEX	CEX	N/A	✗	✗	✓	✓
HTLC-based	Miners	N/A	✓	✗	✓	✓
AMM-based	Smart contract	LPs	✓	✗	✗	✗
Ours	Smart contract	Watchers	✓	✓	✓	✓

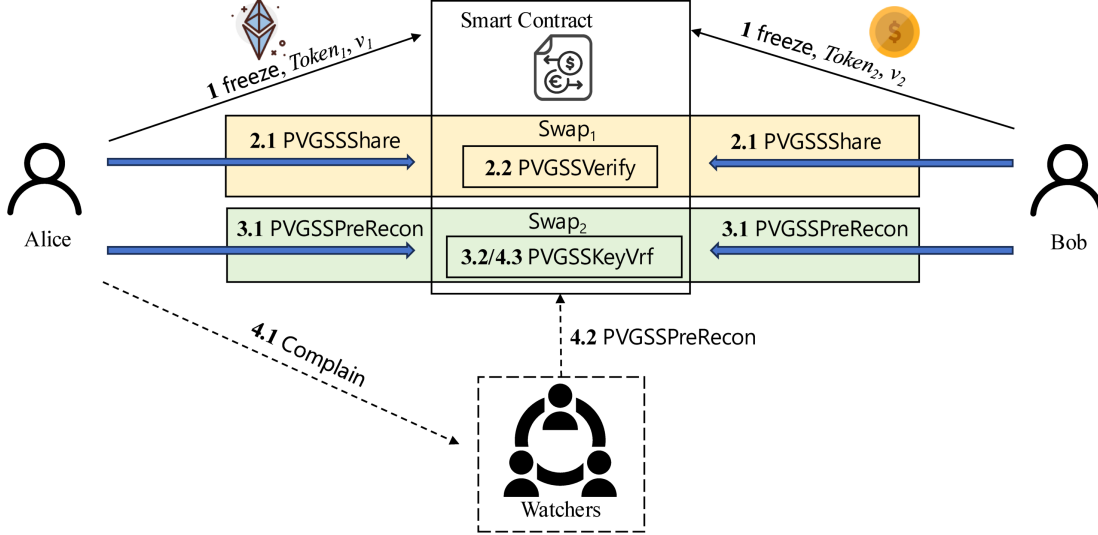


Fig. 10: Overview of the DEX between two exchangers (i.e., Alice and Bob) with passive watchers

Take two exchangers, i.e., Alice and Bob, and n watchers as an example. As shown by Figure 10, the DEX optimistically runs in two communication rounds for Alice and Bob. In the first round, swap₁, each exchanger commits to a secret using PVGSSShare, where all the $n + 2$ entities are considered as shareholders. The correctness of the commitment is guaranteed by the PVGSSVerify algorithm. Refer to Section VI-B for the detailed design of access structure. In the second round, swap₂, each exchanger reveals its decrypted share using PVGSSPreRecon. The correctness of share decryption is ensured by PVGSSKeyVrf. Then, both Alice and Bob jointly recover each other's secrets using PVGSSRecon.

In the pessimistic occasion where a player complains to the watchers, who will be involved to resolve dispute using PVGSSPreRecon. Note that the access structure of the DEX not only tolerates a faulty exchanger but also tolerates $n - t$ faulty watchers.

To promote the prosperity of the DEX, an incentive and penalty mechanism of all the entities should be fully considered. The fundamental assumption is that everyone is rational, and everyone acts in pursuit of profit. Hence, we devise different incentive policies for all occasions in Section VI-D.

B. Access Structure Design

In this section, we design an access structure, which is input to the PVGSSShare algorithm, for the proposed DEX. Suppose the two exchangers are Alice and Bob and the n watchers are $W_1, \dots, W_n$. Figure 11 illustrates the concrete access structure incorporating threshold gates, defined as $(2 \text{ of } (Alice, Bob, (t \text{ of } (W_1, W_2, \dots, W_n))))$. Based on Corollary 1, we can infer that the set $[Alice, Bob]$ is an authorized set. Apparently, if both Alice and Bob are honest, they can recover any secrets of PVGSS instance.

Without loss of generality, if Bob is absent in a PVGSS instance, Alice and at least t watchers can collectively recover the secret. This shows the feature of fault tolerance, i.e., the access structure allows fault of one exchanger and $n - t$ watchers. By Lemma 3, we argue that t can be set to 1 without affecting fairness.

Figure 11 shows the simplest access structure in building a DEX. A DEX can, in fact, adopt a more sophisticated access structure, allowing different watchers to hold varying weights or privileges. This enables the application of Game Theory to incentivize profit-driven competition effectively. That will be future work and it is out of discussion in this paper.

C. Functionalities on Smart Contract

Figure 12 describes the functionality $\mathcal{F}_{DEX}$ defined in smart contract, which interacts with $n + 2$ entities (i.e., 2 exchangers and $n \geq 1$ watchers) in session id. For convenience of description, denote the two exchangers as Alice and Bob. Both Alice

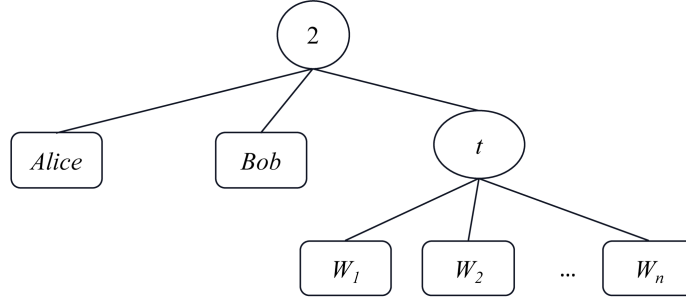


Fig. 11: The access structure leveraged by Alice and Bob, i.e., $\mathbb{A} = (2 \text{ of } (Alice, Bob, (t \text{ of } (W_1, W_2, \dots, W_n))))$. Evidently, under this structure, the number of watchers satisfies $n = |\mathbb{A}| - 2$.

and Bob leverages the same access structure $\mathbb{A}$, as described by Figure 11. Suppose the n watchers are randomly chosen from the registered arbitrageurs for each id. Two expiration time Δt_1 and Δt_2 are indicators for distinguishing the optimistic and pessimistic cases. Appendix A illustrates the primary data structures and global variables/events in implementing $\mathcal{F}_{DEX}$.

Functionality $\mathcal{F}_{DEX}$

For session id, the contract manages two expiration time Δt_1 and $\Delta t_2 > \Delta t_1$, a termination variable *state*.

- Upon receiving (register, id, pk_x) from an entity, store the public key pk_x of the entity x .
- Upon receiving (freeze, id, $Token, v$) from Alice or Bob, freeze v token *Token* in smart contract.
- Upon receiving (withdraw, id) from Alice or Bob, if *state* = failure and current time $ts > \Delta t_2$, transfer the v frozen token *Token* to the sender. Otherwise, abort.
- Upon receiving (swap₁, id, $\{C_i\}$, prf_s) from Alice or Bob, invoke $PVGSSVerify(\{C_i\}, prf_s, \mathbb{A})$. If it returns 0, abort. Otherwise, store $\{C_i\}$.
- Upon receiving (swap₂, id, S_x) from an entity, invoke $PVGSSKeyVrf(C_x, S_x, prf_{S_x}, pk_x)$. If it returns 0, abort. Otherwise, store S_x and notify exchangers.
- Upon receiving (complain, id) from a complainer (i.e., Alice or Bob), if current time $ts > \Delta t_1$, and both Alice and Bob have invoked swap₁ honestly, then notice all watchers. (Each watcher P_i sends the (swap₂, id, S_i) message for the complainer.)
- Upon receiving (determine, id) by anyone, set *state* = success and transfer Alice/Bob's token to Bob/Alice' account, if one of below conditions is satisfied before Δt_2 :
 - Both Alice and Bob have invoked swap₂ honestly.
 - Either Alice or Bob has invoked swap₂ honestly, and at least t watchers have invoked swap₂ honestly.
 Otherwise, set *state* = failure and send back tokens to Alice/Bob's account.

Fig. 12: The functionality $\mathcal{F}_{DEX}$ in smart contract

Suppose Alice, Bob and n watchers have registered their public keys via the $register(id, pk_i)$ interface. Without loss of generality, Alice lists v_1 amount of $Token_1$ for sale and expects to exchange v_2 amount of $Token_2$.⁹ Alice and Bob freeze their tokens by $freeze(id, Token_1, v_1)$ and $freeze(id, Token_2, v_2)$, respectively.

Bob invokes $swap_1(id, \{C_{iB}\}, prf_{s_B})$, where $(\{C_{iB}\}, prf_{s_B}) \leftarrow PVGSSShare(\mathbb{A}, \{pk_i\})$ for secret $s_B \xleftarrow{R} \mathbb{Z}_q$. If Bob is honest (i.e., $\{C_{iB}\}$ is correctly stored in smart contract), Alice then sends $swap_1(id, \{C_{iA}\}, prf_{s_A})$ and $swap_2(id, S_A)$ simultaneously to $\mathcal{F}_{DEX}$, where $(S_A, prf_{S_A}) \leftarrow PVGSSPreRecon(C_A, sk_A)$. If Alice is honest (i.e., $\{C_{iA}\}$ and S_A are recorded in smart contract), Bob invokes $swap_2(id, S_B)$.

To this end, if both exchangers are honest, Alice (or Bob) could recover the Bob (or Alice) secret value hidden in $\{C_{iB}\}$ (or $\{C_{iA}\}$) by $g^{s_B} \leftarrow PVGSSRecon(\mathbb{A}, (S_A, S_B))$ (or g^{s_A}). Hence, they reach a consensus on the swap and anyone can invoke $determine(id)$ of $\mathcal{F}_{DEX}$ to terminate the exchange. Note that though S_A and S_B are published on blockchain and secrets g^{s_A} and g^{s_B} are computable by anyone, the exchange does not reveal individual private keys.

Without loss of generality, we consider Bob is malicious in the pessimistic cases.

- If Bob does not execute swap₁ honestly, Alice will wait to time Δt_2 with failure termination. The complain function in $\mathcal{F}_{DEX}$ guarantees that Bob cannot complain to watchers if he has not invoked swap₁ honestly.

⁹Obviously, $v_1 \cdot Price_{Token_1} \approx v_2 \cdot Price_{Token_2}$. We assume that exchangers are rational and they can obtain Token prices from other exchanges or websites.

- Or if Bob does not execute swap_2 honestly in Δt_1 , Alice can initiate $\text{complain}(\text{id})$ to notify watchers. Each watcher P_i will generate his PVGSS share $(S_i, \text{prf}_{S_i}) \leftarrow \text{PVGSSPreRecon}(C_i, \text{sk}_i)$ and upload it to $\mathcal{F}_{DEX}$ via $\text{swap}_2(\text{id}, S_i)$. With at least t pieces of $\{S_i\}$ collected, Alice can obtain $g^{SB} \leftarrow \text{PVGSSRecon}(\mathbb{A}, (S_A, \{S_i\}))$, representing a success termination. In case not enough watchers provide services, the swap will be failure and faulty watchers will be punished.

D. Incentive Mechanism

Figure 13 presents the mindmap to illustrate all cases of results and the corresponding incentive policy a swap instance. A swap instance ends with either success or failure definitely. Specifically, the success can be determined before Δt_2 and the failure is determined only after Δt_2 . All entities are required to make stakes in the system. The number of swap instances a watcher can serve simultaneously is proportional to the amount of its stakes.

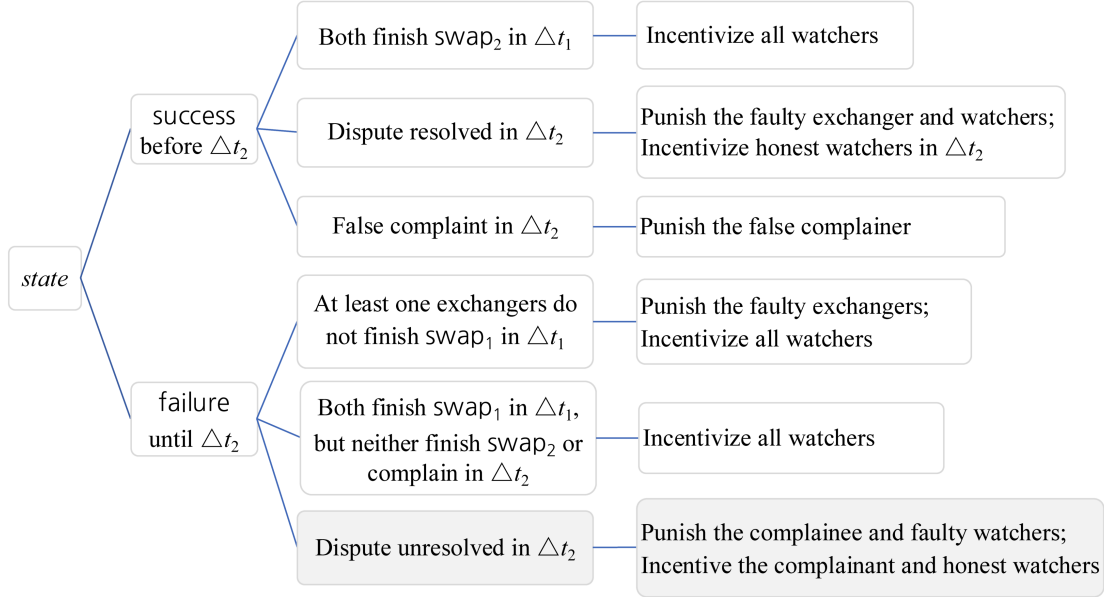


Fig. 13: The incentive and penalty mechanism

The success result includes three cases. The first case is optimistic scenario where both exchangers honestly invoke swap_1 and swap_2 before Δt_1 . In this case, the watchers are not involved and each of them is rewarded with a small tip. The second case shows the scenario where one exchanger does not execute swap_2 before Δt_2 . Then, the other exchanger notice the n watchers to resolve the dispute. In this case, faulty exchanger and watchers are punished to reward the honest watchers who send swap_2 before Δt_2 . The third case introduces the scenario where some exchanger may propose false complaint. In this case, the complainer will be punished.

The failure results also contains three cases. The first case means at least one exchangers give up in Δt_1 . Hence, the faulty exchanger will be punished and all watchers are rewarded. The second case means both exchangers negotiate to abort the swap instance before Δt_2 , then all watchers are rewarded. The third case is an unexpected scenario, where honest complaint is raised but unresolved due to insufficient honest watchers. We impose penalties on the complaine and faulty watchers to compensate the complainant and honest watchers.

E. Security Analysis of the DEX

Before describing the achieved properties, we argue the potential threats to the DEX has little impacts.

Lemma 3 (Dos attack). *An exchanger or (even up to n) watchers, who abort the protocol, can not bias the fairness.*

Proof. As described in Section VI-C, if an exchanger aborts without executing swap_1 or more than $n - t$ watchers abort without providing arbitration, the swap instance ends with failure; Otherwise, the instance ends with success. Fairness is guaranteed according to the protocol termination result and only faulty entities will bear specific loss. Thus, it is feasible to set $t = 1$ with arbitrary $n \geq 1$ in practice. $\square$

Lemma 4 (Collusion/Coercion attack). *An exchanger colluding with (even up to n) malicious watchers cannot break the fairness.*

Proof. The exchanging tokens are locked in the smart contract until termination. Both the exchange protocol and smart contracts are transparent and publicly verifiable. The final *state* of an exchange instance is either success or failure, determined

no later than Δt_2 . If $state = success$, the locked tokens are transferred to the counterparties' accounts; otherwise, they are returned to the respective exchangers' accounts. Consequently, even if an exchanger colludes with all n watchers, they cannot gain any advantage over the smart contract. Furthermore, inactive or malicious entities are penalized. $\square$

Lemma 5 (Sybil attack). *An arbitrageur cannot gain disproportionate profits through generating fake identities.*

Proof. An arbitrageur can be selected as a watcher in proportion to the amount of currency they have deposited. This implies that the nothing-at-stake attack is also eliminated. If a swap instance is ongoing, a proportional amount of deposited currency is locked, rendering a Sybil attack meaningless. $\square$

Below, we prove the security properties by Claims. The security of DEX is informally presented, as it directly applies PVGSS algorithms within two rounds.

Claim 1 (Decentralization). *No trusted manager is required to execute the protocol or manage participants.*

Proof. As designed, anyone can register with deposited digital currency as an exchanger or an arbitrageur. Anyone is equal before the smart contract. By Lemma 5, we prove that it is infeasible for arbitrageurs to gain additional profits by generating fake identities. $\square$

Claim 2 (Fairness). *Either exchangers acquire the counterpart's token or neither acquires.*

Proof. According to the determine functionality in $\mathcal{F}_{DEX}$, the $state$ of a swap instance is determined as either success or failure based on the actions of exchangers and watchers. A success results in a successful exchange, while a failure ensures the return of tokens to exchangers, guaranteeing fairness. Additionally, as demonstrated in Lemma 3 and Lemma 4, neither DoS attacks nor collusion attacks affect fairness. $\square$

Claim 3 (Termination). *An exchanging instance should be terminated at specific time point.*

Proof. A swap instance can terminate with success at any time before Δt_1 if both exchangers act honestly. The parameter Δt_1 allows exchangers to file complaints through the smart contract. And the complaints should be handled in Δt_2 by the watchers. Consequently, the swap instance is guaranteed to reach a definitive termination by Δt_2 , regardless of the presence of faulty entities. $\square$

Claim 4 (Arbitration with Optimism and Statelessness). *The watchers can resolve dispute and they are not involved if no disputes arise.*

Proof. As the access structure designs (cf. Section VI-B), if an exchanger aborts in $swap_2$, at least t watchers can be of help in guaranteeing fairness with success termination. As describe in Section VI-C, the swap instance can normally ends with success if both exchangers are honest and watchers are not involved, showing the optimism of them. Moreover, watchers can take action according to smart contract's status without requiring additional storage for each swap instance, demonstrating their statelessness. $\square$

Claim 5 (Accountability). *Dishonest behaviors can be detected and faulty entities is punished.*

Proof. The whole protocol is designed through smart contract and all operations are publicly verifiable by any third party. Specifically, the public verifiability is ensured by Theorem 2 and Theorem 3. Therefore, it is feasible to find out faulty entities and punish them by confiscating a certain percentage of their deposits. $\square$

Claim 6 (Incentive Compatibility). *The protocol comprehensively considers interests of exchangers and watchers, forcing them to behave honestly.*

Proof. By Claim 3, we show that the $state$ is determined no later than Δt_2 . As illustrated in Section VI-D, all possible outcomes for the $state$ values and the corresponding incentives are considered. Besides, the protocol considers both exchangers and arbitrageurs, who are accountable by Claim 5. Thus, we can design sophisticated incentive mechanisms to encourage honest behavior and penalize faulty actions, promoting the adoption of the DEX. Figure 13 illustrates the detailed incentive mechanisms. $\square$

VII. EXPERIMENTAL EVALUATION

We implement our PVGSS scheme using Golang (off-chain) and Solidity (on-chain). The method in [24] supports only boolean formula-based access structures; therefore, we adopt the approach proposed in [69] to transform a general access structure into a LSSS matrix. To make on-chain and off-chain operations compatible, we choose BN128 as the asymmetric elliptic curve group, which is officially supported by Ethereum. The benchmarks of PVGSS schemes are executed on a Intel (R) Core (TM) i7-11800H @2.30GHz with 4GB RAM running Ubuntu 22.04.5 LTS and Golang1.22.0. The smart contract codes are deployed and tested on Ethereum Sepolia testnet. The proof of concept implementation is available on GitHub¹⁰.

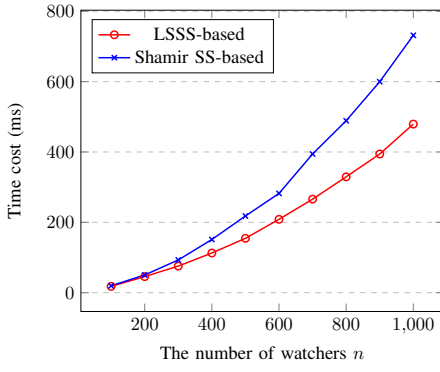


Fig. 14: Cost of PVGSSShare

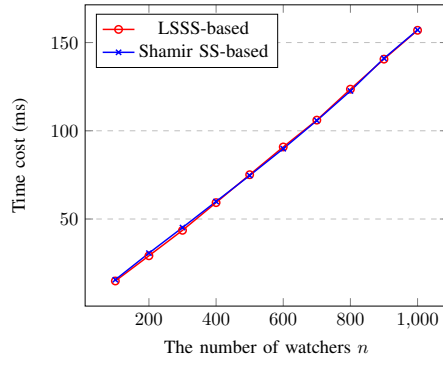


Fig. 15: Cost of PVGSSVerify

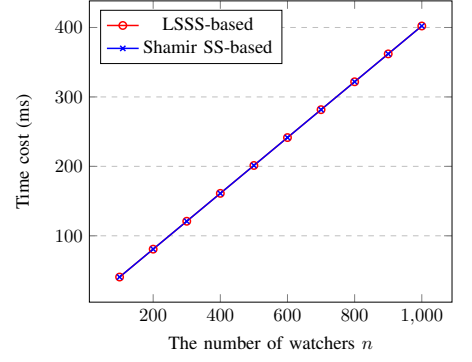


Fig. 16: Cost of n times of PVGSSKeyVrf

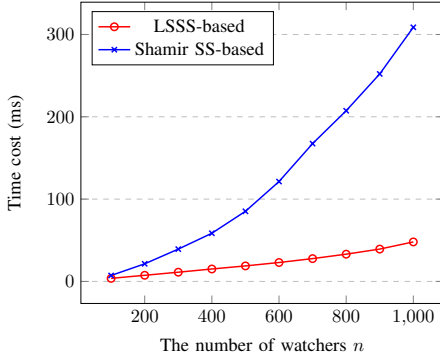


Fig. 17: PVGSSRecon with an authorized set size $t + 1$

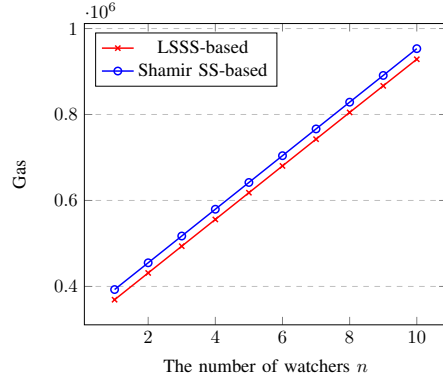


Fig. 18: On-chain gas cost of $\text{swap}_1 + \text{swap}_2$

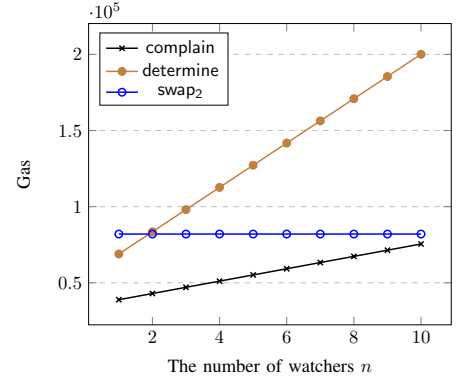


Fig. 19: On-chain cost of complain, determine, swap_2

We choose the access structure designed for the DEX (cf. Figure 11) to evaluate the performance of the PVGSS scheme, where $t = n/2 + 1$ and n ranges from 100 to 1000. Figure 14-17 show the off-chain computational overheads of the PVGSS schemes. The cost of PVGSSShare and PVGSSRecon are super-linear, due to polynomial evaluation and Lagrange interpolation in Shamir SS-based approach and matrix multiplication in LSSS-based approach. Besides, the PVGSSPreRecon algorithm costs about $418\mu\text{s}$ for calculation of each S_i . It can be seen all algorithms can be calculated within seconds given $n = 1000$, indicating practicality of the PVGSS schemes in large-scale systems.

Further, we evaluate the gas consumption of the DEX. As proved in Lemma 3, we can set $t = 1$ and arbitrary $n \geq 1$ for the DEX in practice. We present the on-chain gas cost with n ranging from 1 to 10. We first introduce the cost in the

¹⁰<https://github.com/scottocs/pvgss>

TABLE III: Monetary cost of the on-chain operations

Operation	Gas cost	USD
register	167767	\$0.358
freeze	71250	\$0.152
swap_1	$n=1$ 310787	\$0.663
	$n=5$ 559855	\$1.195
	$n=9$ 808882	\$1.726
swap_2	82020	\$0.175
total, i.e.,	$n=1$ 392807	\$0.838
$\text{swap}_1 + \text{swap}_2$ (Shamir SS-based)	$n=5$ 813639	\$1.37
	$n=9$ 1062603	\$1.901
complain	$n=1$ 38894	\$0.083
	$n=5$ 55166	\$0.118
	$n=9$ 71414	\$0.152
determine	$n=1$ 68917	\$0.147
	$n=5$ 127209	\$0.271
	$n=9$ 185499	\$0.396

normal case, where both exchangers honestly abide by the protocol. In swap_1 , calculation of PVGSSVerify and storage of $\{C_i\}$ are included. The swap_2 , which invokes PVGSSKeyVrf , roughly costs a constant amount gas (i.e., 82020) for each exchanger. The PVGSSKeyVrf algorithm is the same for both LSSS-based and Shamir SS-based PVGSS , hence they cost equal gas consumption. Figure 18 depicts the gas consumption of the $\text{swap}_1 + \text{swap}_2$ on smart contract. It can be observed that the LSSS-based PVGSSShare is slightly more efficient, as matrix multiplication is somewhat more computationally intensive than Lagrange interpolation in Solidity. Then, we depict the gas cost in exceptional scenarios where arbitration procedures are required. Particularly, an exchanger calls complain , then each watcher invokes swap_2 and finally anyone can execute determine . Figure 19 shows the gas cost of functionalities of complain , determine and swap_2 .

As of the time of writing (March 18, 2025), the median gas price is 1.1 gwei, and the price of ETH is 1,940 USD. Table III presents the monetary cost of each operation on the smart contract. For convenience, only the Shamir SS-based PVGSS is evaluated. The gray cells highlight the optimistic case, where the cost for each exchanger is only \$0.838/\$1.37/\$1.901 for $n = 1/5/9$ and $t = 1$. If watchers are involved in the pessimistic case, the complaint cost is less than 0.2 USD, and each watcher incurs a cost of approximately 0.175 USD (as swap_2 requires).

VIII. CONCLUSION

We begin by introducing the concept of publicly verifiable generalized secret sharing (PVGSS) and compare it with other SS-related schemes (i.e., LSSS, GSS, VSS, PVSS and CPABE). The comparison highlights that PVGSS not only supports generalized access structure, but also facilitates public verifiability, making it ideal primitive for transparent systems like blockchain. Then we propose two constructions with unified description: one is Shamir-based and the other is LSSS-based. Rigorous proofs are given to prove the security proofs of the constructions. Furthermore, we build a PVGSS -based decentralized exchange (DEX), which guarantees optimistic fairness and passive arbitration due to the sophisticated designed access structure. Comprehensive experiments and detailed analysis demonstrates the feasibility of the DEX in real-world applications. Looking ahead, we plan to further explore PVGSS -related research. First, we aim to implement additional applications discussed in Section I-A and beyond. Second, we intend to formalize verifiable generalized secret sharing (VGSS , akin to VSS [15]) and examine its constructions. Lastly, we will explore variations of PVGSS schemes, such as proactive PVGSS or q- PVGSS , akin to q-PVSS [12].

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China for Young Scientists (No. 62302129), National Natural Science Foundation of China (Nos. 62272107), Hainan Province Key R&D plan project (No. ZDYF2024GXJS030) and the HK RGC General Research Fund (Nos. 16208120 and 16214121).

REFERENCES

- [1] Shamir, A. How to share a secret. *Comm. of the ACM*, 1979, 22(11), pp. 612-613.
- [2] Das, S., Yurek, T., Xiang, Z., Miller, A., Kokoris-Kogias, L. and Ren, L. Practical asynchronous distributed key generation. In *S&P*, 2022, pp. 2518-2534.
- [3] Choi, K., Manoj, A. and Bonneau, J. Sok: Distributed randomness beacons. In *S&P*, 2023, pp. 75-92.
- [4] Sahai, A., and Waters, B. Fuzzy identity-based encryption. In *Eurocrypt*, 2005, pp. 457-473.
- [5] Rathee, M., Shen, C., Wagh, S. and Popa, R.A. Elsa: Secure aggregation for federated learning with malicious actors. In *S&P*, 2023, pp. 1961-1979.
- [6] Dauterman, E., Rathee, M., Popa, R.A. and Stoica, I. Waldo: A private time-series database from function secret sharing. In *S&P*, 2022, pp. 2450-2468.
- [7] Feneuil, T. and Rivain, M. Threshold linear secret sharing to the rescue of MPC-in-the-head. In *Asiacrypt*, 2023, pp. 441-473.
- [8] Basu, S., Tomescu, A., Abraham, I., Malkhi, D., Reiter, M.K. and Sirer, E.G. Efficient verifiable secret sharing with share recovery in BFT protocols. In *CCS*, 2019, pp. 2387-2402.
- [9] Karchmer, M. and Wigderson, A. On span programs. In *Annual Structure in Complexity Theory Conference*, 1993, pp. 102-111.
- [10] Feldman, P. A practical scheme for non-interactive verifiable secret sharing. In *FOCS*, 1987, pp. 427-438.
- [11] Zhang, Z., Li, W., Guo, Y., Shi, K., Chow, S.S., Liu, X. and Dong, J. Fast RS-IOP Multivariate Polynomial Commitments and Verifiable Secret Sharing. In *USENIX Security*, 2024, pp. 3187-3204.
- [12] Cascudo, I., and David, B. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In *Eurocrypt*, 2024, pp. 216-248.
- [13] Cascudo, I., and David, B. SCRAPE: Scalable randomness attested by public entities. In *ACNS*, 2017, pp. 537-556.
- [14] Kate, A., Zaverucha, G.M. and Goldberg, I. Constant-size commitments to polynomials and their applications. In *Asiacrypt*, 2010, pp. 177-194.
- [15] Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S. and Stern, G. Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In *Crypto*, 2023, pp. 39-70.
- [16] Stadler, M. Publicly verifiable secret sharing. In *Eurocrypt*, 1996, pp. 190-199.
- [17] Ito, M., Saito, A. and Nishizeki, T. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan*, 1989, 72(9), pp. 56-64.
- [18] Benaloh, J., and Leichter, J. Generalized secret sharing and monotone functions. In *Crypto*, 1990.
- [19] Yu, S., Wang, C., Ren, K. and Lou, W. Achieving secure, scalable, and fine-grained data access control in cloud computing. In *INFOCOM*, 2010, pp. 1-9.
- [20] Xu, S., Yang, G., Mu, Y. and Deng, R.H. Secure fine-grained access control and data sharing for dynamic groups in the cloud. *IEEE TIFS*, 2018, 13(8), pp. 2101-2113.
- [21] Garg, S., Jain, A., Mukherjee, P., Sinha, R., Wang, M. and Zhang, Y. Cryptography with weights: MPC, encryption and signatures. In *CRYPTO*, 2023, pp. 295-327.
- [22] Solanki, N., Huang, Y., Yen, I.L., Bastani, F. and Zhang, Y. Resource and role hierarchy based access control for resourceful systems. In *COMPSAC*, pp. 480-486.
- [23] Bethencourt, J., Sahai, A., and Waters, B. Ciphertext-Policy Attribute-Based Encryption. In *S&P*, 2007, pp. 321-334.

- [24] Lewko, A., and Waters, B. Decentralizing attribute-based encryption. In *Eurocrypt*, 2011, pp. 568-588.
- [25] Lin, C., Hu, H., Chang, C.C. and Tang, S. A publicly verifiable multi-secret sharing scheme with outsourcing secret reconstruction. *IEEE Access*, 2018, 6, pp. 70666-70673.
- [26] Okamoto, T. and Takashima, K. Decentralized attribute-based signatures. In *PKC*, 2013, pp. 125-142.
- [27] Gilad, Y., Hemo, R., Micali, S., Vlachos, G. and Zeldovich, N. Algorand: Scaling byzantine agreements for cryptocurrencies. In *SOSP*, 2017, pp. 51-68.
- [28] Scafuro, A., Siniscalchi, L. and Visconti, I. Publicly verifiable proofs from blockchains. In *PKC*, 2019, pp. 374-401.
- [29] Beimel, A. Secret-Sharing Schemes for General Access Structures: An Introduction. *Cryptology ePrint Archive*. 2025.
- [30] Huber, N., Küsters, R., Krips, T., Liedtke, J., Müller, J., Rausch, D., Reisert, P. and Vogt, A. Kryvos: Publicly tally-hiding verifiable e-voting. In *CCS*, 2019, pp. 1443-1457.
- [31] Baum, C., Orsini, E., Scholl, P. and Soria-Vazquez, E. Efficient constant-round MPC with identifiable abort and public verifiability. In *Crypto*, 2020, pp. 562-592.
- [32] Szydło, M., 2004. Merkle tree traversal in log space and time. In *Eurocrypt*, 2004, pp. 541-554.
- [33] Kemmoe, V.Y. and Lysyanskaya, A., 2024, December. RSA-Based Dynamic Accumulator without Hashing into Primes. In *CCS*, 2024, pp. 4271-4285.
- [34] Schnorr, C.P., 1990. Efficient identification and signatures for smart cards. In *Crypto*, 1990, pp. 239-252.
- [35] Setty, S. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Crypto*, 2020, pp. 704-737.
- [36] Polygon. Polygon zkEVM. <https://docs.polygon.technology/zkEVM/>.
- [37] Drăgan, C.C. and Tiplea, F.L., . Distributive weighted threshold secret sharing schemes. *Information sciences*, 2016, 339, pp. 85-97.
- [38] Beimel, A., Tassa, T., and Weinreb, E. Characterizing ideal weighted threshold secret sharing. In *TCC*, 2005, pp. 600-619.
- [39] Chen, Q., Tang, C. and Lin, Z. Efficient explicit constructions of multipartite secret sharing schemes. *IEEE TIT*, 2021, 68(1), pp. 601-631.
- [40] Tassa, T., and Dyn, N. Multipartite secret sharing by bivariate interpolation. *JoC*, 2009, 22(2), pp. 227-258.
- [41] Simmons, G.J., 1988, August. How to (really) share a secret. In *Eurocrypt*, 1988, pp. 390-448.
- [42] Damgård, I. On Σ -protocols. *Lecture Notes, University of Aarhus, Department for Computer Science*, 2002.
- [43] Fiat, A., and Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In *Eurocrypt*, 1986, pp. 186-194.
- [44] Tassa, T. Hierarchical threshold secret sharing. *JoC*, 2007, 20(2), pp. 237-264.
- [45] Chattopadhyay, A.K., Saha, S., Nag, A. and Nandi, S. Secret sharing: A comprehensive survey, taxonomy and applications. *Computer Science Review*, 2024, 51, 100608.
- [46] Beimel, A. Secret-sharing schemes: A survey. In *ICC*, 2011, pp. 11-46.
- [47] Cascudo, I., David, B., Garms, L., and Konring, A. YOLO YOSO: fast and simple encryption and secret sharing in the YOSO model. In *Asiacrypt*, 2022, pp. 651-680.
- [48] Cascudo, I., and David, B. ALBATROSS: publicly attestable batched randomness based on secret sharing. In *Asiacrypt*, 2020, pp. 311-341.
- [49] Fujisaki, E., and Okamoto, T. A practical and provably secure scheme for publicly verifiable secret sharing and its applications. In *Eurocrypt*, 1998, pp. 32-46.
- [50] Schoenmakers, B. A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic. In *Crypto*, 1999, pp. 148-164.
- [51] Ruiz, A., and Villar, J. L. Publicly verifiable secret sharing from Paillier's cryptosystem. In *SAC*, 2005.
- [52] Erwig, A., Faust, S., Hostáková, K., Maitra, M. and Riahi, S. Two-party adaptor signatures from identification schemes. In *PKC*, 2021, pp. 451-480.
- [53] Thyagarajan, S.A., Malavolta, G. and Moreno-Sanchez, P. Universal atomic swaps: Secure exchange of coins across all blockchains. In *S&P*, 2022, pp. 1299-1316.
- [54] Zhang, L., Kan, H., Qiu, F. and Hao, F. A Publicly Verifiable Optimistic Fair Exchange Protocol Using Decentralized CPABE. *CJ*, 2024, 67(3), pp. 1017-1029.
- [55] Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostáková, K., Maffei, M., Moreno-Sanchez, P. and Riahi, S. Generalized channels from limited blockchain scripts and adaptor signatures. In *Asiacrypt*, 2021, pp. 635-664.
- [56] Augusto, A., Belchior, R., Correia, M., Vasconcelos, A., Zhang, L. and Hardjono, T. Sok: Security and privacy of blockchain interoperability. In *S&P*, 2024, pp. 3840-3865.
- [57] Xu, J., Paruch, K., Cousaert, S. and Feng, Y. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Comp. Surv.*, 2023, 55(11), pp. 1-50.
- [58] Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L. and Juels, A. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *S&P*, 2020, pp. 910-927.
- [59] Baum, C., David, B. and Frederiksen, T.K. P2DEX: privacy-preserving decentralized cryptocurrency exchange. In *ACNS*, 2021, pp. 163-194.
- [60] Heimbach, L., Schertenleib, E. and Wattenhofer, R. Risks and returns of uniswap v3 liquidity providers. In *AFT*, 2022, pp. 89-101.
- [61] Park, S., Bastani, O. and Kim, T. ACon2: Adaptive Conformal Consensus for Provable Blockchain Oracles. In *USENIX Security*, 2023, pp. 3313-3330.
- [62] Xie, T., Zhang, J., Cheng, Z., Zhang, F., Zhang, Y., Jia, Y., Boneh, D. and Song, D. zkbridge: Trustless cross-chain bridges made practical. In *CCS*, 2022, pp. 3003-3017.
- [63] Li, Y., Liu, H. and Tan, Y. POLYBRIDGE: A crosschain bridge for heterogeneous blockchains. In *ICBC*, 2022, pp. 1-2.
- [64] Dziembowski, S., Faust, S. and Hostáková, K. General state channel networks. In *CCS*, 2018, pp. 949-966.
- [65] Gudgeon, L., Moreno-Sanchez, P., Roos, S., McCorry, P. and Gervais, A. Sok: Layer-two blockchain protocols. In *FC*, 2020, pp. 201-226.
- [66] Benhamouda, F., Gentry, C., Gorbunov, S., Halevi, S., Krawczyk, H., Lin, C., Rabin, T. and Reyzin, L. Can a public blockchain keep a secret?. In *TCC*, 2020, pp. 260-290.
- [67] Goyal, V., Kothapalli, A., Masserova, E., Parno, B. and Song, Y. Storing and retrieving secrets on a blockchain. In *PKC*, 2022, pp. 252-282.
- [68] Cerulli, A., Connolly, A., Neven, G., Preiss, F.S. and Shoup, V. vetkeys: How a blockchain can keep many secrets. *Cryptology ePrint Archive*. 2023.
- [69] Liu, Z., Cao, Z. and Wong, D.S. Efficient generation of linear secret sharing scheme matrices from threshold access trees. *Cryptology ePrint Archive*. 2010
- [70] Chaum, D., & Pedersen, T. P. Wallet databases with observers. In *CRYPTO*, 1992, pp. 89-105.
- [71] Bagherpour, B., 2022. An efficient publicly verifiable and proactive secret sharing scheme. *Adv. in Math. of Comm.*, 16(4), pp. 671-691.
- [72] Pagnia, H. and Gärtner, F.C. On the impossibility of fair exchange without a trusted third party. Technical Report TUD-BS-1999-02, Darmstadt University of Technology, Department of Computer Science, Darmstadt, Germany. 1999.

APPENDIX

A. Global States in $\mathcal{F}_{DEX}$

```

1 struct Order {
2     address seller; //Order creator
3     address tokenSell; // Token to sell (e.g., ETH)
4     uint256 amountSell; // Amount to sell (e.g., 2 ETH)
5     address tokenBuy; // Token to buy (e.g., USDT)
6     uint256 amountBuy; // Amount to buy (e.g., 7000 USDT)

```

```

7     bool isActive;        // Order state
8 }
9 // Store orders
10 mapping(uint256 => Order) public orders;
11
12 // values to track session state
13 enum SessionState { Active, halfSwap1, finishSwap1, halfSwap2, Complain, Success, Failure }
14 struct Session {
15     SessionState state; // Session state
16     address[] exchangers; // seller as exchanger[0], buyer as exchanger[1] in the session
17     address[] watchers; // Watchers in the session
18     mapping(address => G1Point) Cshares1; //shares from seller
19     mapping(address => G1Point) Cshares2; //shares from buyer
20     mapping(address => G1Point) shares; //decrypted shares
21     uint256 t1; // Expiration time t1
22     uint256 t2; // Expiration time t2
23     bool[2] seller_flag; // swap flag of seller
24     bool[2] buyer_flag; // swap flag of buyer
25     mapping(address => bool) watcher_flag; //submit flag of watcher
26 }
27 // Store sessions
28 mapping(uint256 => Session) public sessions;
29
30 //Events in the smart contract
31 event TokensReceived(address token, address from, uint256 amount);
32 event TokensFrozen(address token, address from, uint256 amount, uint256 sessionId);
33 event TokensSwapped(address token, address from, uint256 amount, uint256 sessionId);
34 event ComplaintFired(address complainer, uint256 sessionId);
35 event SessionStateUpdated(uint256 sessionId, SessionState state);
36 event UserNotified(uint256 sessionId, address user);
37 event OrderCreated(uint256 orderId, address seller, address tokenSell, uint256 amountSell,
    address tokenBuy, uint256 amountBuy);
38 event SessionCreated(uint256 orderId, address seller, address buyer, address[] watchers,
    uint256 t1, uint256 t2);
39 event Incentivized(address user, uint256 amount);
40 event Penalized(address user, uint256 amount);

```